# Comparative Analysis of Machine Learning and Hybrid Deep Learning Models for Cardiovascular Disease Prediction

*Report submitted to the SASTRA Deemed to be University*
*as the requirement for the course*

**INT300 - MINI PROJECT**


*Submitted by*
**KONTHAM MADHAV RAM SAI**
(Reg. No.: 127015050, B.Tech IT)
**PANDANA BOYANA KUMAR**
(Reg. No.: 127015068, B.Tech IT)
**YARAMALA SAI KISHORE REDDY**
(Reg. No.: 127014065, B.Tech ICT)


**May 2026**



**SCHOOL OF COMPUTING**
**THANJAVUR, TAMIL NADU, INDIA – 613 401**

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act,1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

## SCHOOL OF COMPUTING

**THANJAVUR - 613 401**

## Bonafide Certificate

This is to certify that the report titled "**Comparative Analysis of Machine Learning and Hybrid Deep Learning Models for Cardiovascular Disease Prediction**" submitted as a requirement for the course, **INT300: MINI PROJECT** for B.Tech., is a bonafide record of the work done by,

**Kontham Madhav Ram Sai** (Reg. No.: 127015050, B.Tech IT), **Pandana Boyana Kumar** (Reg. No.: 127015068, B.Tech IT) and **Yaramala Sai Kishore Reddy** (Reg. No.: 127014065, B.Tech ICT) during the academic year 2025-26, in the School of Computing, under my supervision.

**Signature of Project Supervisor:**

**Name with Affiliation** : Dr. L. Gowri, Asst. Professor-II,
School of Computing, SASTRA Deemed University

**Date** : May 3, 2026

Mini Project Viva voice held on _____

**Examiner 1** **Examiner 2**

# Acknowledgements

We would like to thank our Honorable Chancellor **Prof. R. Sethuraman** for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honorable Vice-Chancellor **Dr. S. Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. V. S. Shankar Sriram**, Dean, School of Computing, **Dr. R. Muthaiah**, Associate Dean, Research, **Dr. K.Ramkumar**, Associate Dean, Academics, **Dr. D. Manivannan**, Associate Dean, Infrastructure, **Dr. R. Alageswaran**, Associate Dean, Students Welfare

Our guide **Dr.Gowri L, Assistant Professor-II**, School of Computing was the driving force behind this whole idea from the start. Her deep insight in the field and invaluable suggestions helped us in making progress throughout our project work. We also thank the project review panel members for their valuable comments and insights which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for providing us an opportunity to showcase our skills through project

# Table of Contents

# List of Figures



# List of Tables

# Abbreviations & Notations

## Abbreviations:

| | |
|---|---|
| CVD | Cardiovascular Disease |
| ML | Machine Learning |
| SVM | Support Vector Machine |
| KNN | K-Nearest Neighbour |
| DT | Decision Tree |
| RF | Random Forest |
| DL | Deep Learning |
| HDNN | Hybrid Deep Neural Network |
| ANN | Artificial Neural Network |
| LSTM | Long Short-Term Memory |
| CNN | Convolutional Neural Network |
| CNN-LSTM | Convolutional Neural Network – Long Short-Term Memory (Hybrid Model) |
| MCC | Matthews Correlation Coefficient |
| ROC | Receiver Operating Characteristic |
| AUC | Area Under Curve |
| ReLU | Rectified Linear Unit |
| RNN | Recurrent Neural Network |
| SMOTE | Synthetic Minority Over-sampling Technique |
| CV | Cross Validation |
| DLNN | Deep Learning Neural Network |

| | |
|---|---|
| BN | Batch Normalization |
| AUC | Area Under Curve |
| ROC | Receiver Operating Characteristic |
| HDNN | Hybrid Deep Neural Network |

## English Symbols:

| | |
|---|---|
| % | Percentage |
| x | Input feature vector |
| y | Output label |
| X | Feature matrix |
| w | Weight vector |
| b | Bias term |
| n | Number of samples |
| k | Number of neighbors (KNN) |
| d | Distance metric |
| O | Output of CNN layer |
| IG | Information Gain |
| G | Gini Index |

## Greek Symbols:

| | |
|---|---|
| σ | Sigmoid activation function |
| tanh | Hyperbolic tangent function |
| μ | Mean of dataset |
| α | Learning rate |
| θ | Model parameters |

# Abstract

Heart disease (HD) is considered one of the main reasons for death worldwide. According to the World Health Organization (WHO), around 17.9 million people lose their lives every year because of it. Detecting heart disease early is quite difficult, but at the same time, it is very important since it allows doctors to take action at the right time and improve the chances of recovery. Because of this, many researchers are now focusing on developing automated systems that can help in predicting heart disease more effectively. In recent years, Deep Learning (DL) methods have shown good results in medical diagnosis because they can learn complex patterns from data.

In this work, a heart disease prediction system is proposed using Hybrid Deep Neural Networks (HDNNs). The main idea is to combine different neural network models so that better features can be learned from the input data. For this purpose, Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), and Long Short-Term Memory (LSTM) models are used. In addition, a hybrid model combining CNN and LSTM with extra dense layers is developed. This combination helps in capturing both spatial and sequential information present in heart disease datasets.

The proposed system is tested on two publicly available datasets, including the Cleveland dataset and a combined dataset consisting of Switzerland, Cleveland, Statlog, Hungarian, and Long Beach VA data.The performance of the models is evaluated by using different measures such as accuracy, precision, sensitivity, specificity, F1-score, Matthews Correlation Coefficient (MCC), and Area Under the Curve (AUC). From the results that were obtained, the proposed hybrid model achieves an accuracy of nearly 98.86%, which is higher when compared to many of the existing methods.

Overall, the results show that combining multiple deep learning models helps in improving the prediction performance. Such a model would prove useful in practical healthcare applications, as it would aid doctors in early detection of heart ailments, as well as assist them in prescribing appropriate medical treatment.

# Table of Contents

# CHAPTER 1

# SUMMARY OF THE BASE PAPER

**Title:** A Robust Heart Disease Prediction System Using Hybrid Deep Neural Networks
**Journal Name:** IEEE Access
**Publisher:** IEEE
**Year:** 2023
**Indexed in: Scopus,** SCI

## 1.1 Introduction

In recent years, the number of patients with Heart Disease (HD) or Cardiovascular Diseases (CVD) has grown tremendously, thus ranking this disorder among the major causes of deaths throughout the world. The sharp growth in the number of heart disease cases poses great difficulties for healthcare providers around the world. The complexity and great variety of heart diseases make conventional approaches ineffective. That is why there is a need for the implementation of innovative technologies that will help specialists make decisions quicker and more precisely.

This project's main goal is the creation of an efficient system of predicting heart diseases by means of advanced deep neural networks in order to help medical practitioners make the right decisions. This paper discusses the key issue of heart disease detection using machine learning and deep learning algorithms with the aim of enhancing the accuracy of diagnosing patients and decreasing mortality rates.

The authors have created a solid prediction framework that depends on hybrid deep neural networks that can help in analyzing diverse medical databases. Such an algorithm will integrate the various types of models like artificial neural networks (ANN), convolutional neural networks (CNN), and long short-term memory (LSTM) neural networks. One of the most significant contributions of the paper is the combination of CNN and LSTM networks for detecting important features in medical databases.

In addition, the researchers test the efficiency of the proposed system and compare its performance with conventional machine learning models, namely logistic regression, decision trees, random forests, naive bayes, and support vector machines. The research shows that the hybrid model yields an accuracy of 98.86%, demonstrating its effectiveness in predicting heart disease accurately.

## 1.2 Problem Statement

The difficulty in predicting the heart disease is because of the existence of complex and non-linear relationships in medical data. It is also common to find traditional machine learning algorithms failing when modeling such types of relationships because of their complexity.

Moreover, the datasets in medicine might have missing values, noise, and even some irrelevant data, which eventually leads to a reduction in the accuracy of predictions. Variations in patient conditions and multiple risk factors also contribute to making classification harder. The high data dimensionality also increases the difficulty of feature selection.
Thus, there is a requirement of introducing the new approach to this problem which can improve the accuracy and support the early detection of heart disease.

## 1.3 Proposed Solution and System Architecture

Traditional heart disease prediction methods depend on clinical observation, which makes them inefficient in handling the complex and high-dimensional medical data. Conventional heart disease predictive analysis makes use of clinical observations and traditional approaches. These approaches have their drawbacks in processing large amounts of complex data in healthcare systems. To overcome this, the base paper proposes an automated predictive system that makes use of the Hybrid Deep Neural Networks (HDNN), which integrates Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) models to improve early diagnosis and decision-making.

**Dataset Description:**
The predictive system makes use of two benchmark datasets obtained from UCI Machine Learning Repository. The first, the Cleveland Heart Disease dataset, consists of 303 observations with 14 features. There are multiple classes in which 0 represents absence of disease and others for the presence of heart disease. The second is a combined dataset, which includes Switzerland, Cleveland, Statlog, Hungarian, and Long Beach VA datasets. It consists of 1190 instances with 11 attributes. Among other important factors, the two datasets contain parameters such as chest pain type, cholesterol level, maximum heart rate, and fasting blood sugar.

**Feature Selection and Extraction:**
In data preprocessing stage, Z-score normalization is used to normalize the data. Feature selection is conducted using Extra Tree Classifier (ETC), which plays a role of ranking the feature importance and reduces dimensionality by selecting the most relevant attributes. In the proposed system, CNN and ANN have been adopted for the extraction of meaningful features and learning complex patterns from datasets. CNN will help in the extraction of important features from the data while ANN can learn complex patterns in non-linear relations.

**Heart Disease Classification:**
The classification is performed using Long Short Term Memory (LSTM) and a hybrid CNN-LSTM (HDNN), which will accurately determine the presence of heart disease as depicted in Fig 1.1. The CNN architecture is utilized to capture high-level and semantically relevant features from the imported medical datasets, which can identify any patterns existing among the attributes in the clinical data. The features identified by the CNN are fed into the LSTM network, in which the sequential and temporal dependencies in the data will be refined by the model.

The CNN-LSTM model is trained using labeled datasets, whereas the outputs are binary classes: either heart disease is diagnosed (1) or heart disease is not found (0).

The Evaluation of model is performed using measures such as accuracy, precision, recall, F1-score, ROC curves, AUC, and MCC. HDNN performs outstandingly in comparison with standalone ANN, CNN, and LSTM models in terms of generalization and efficiency, thus increasing prediction accuracy.

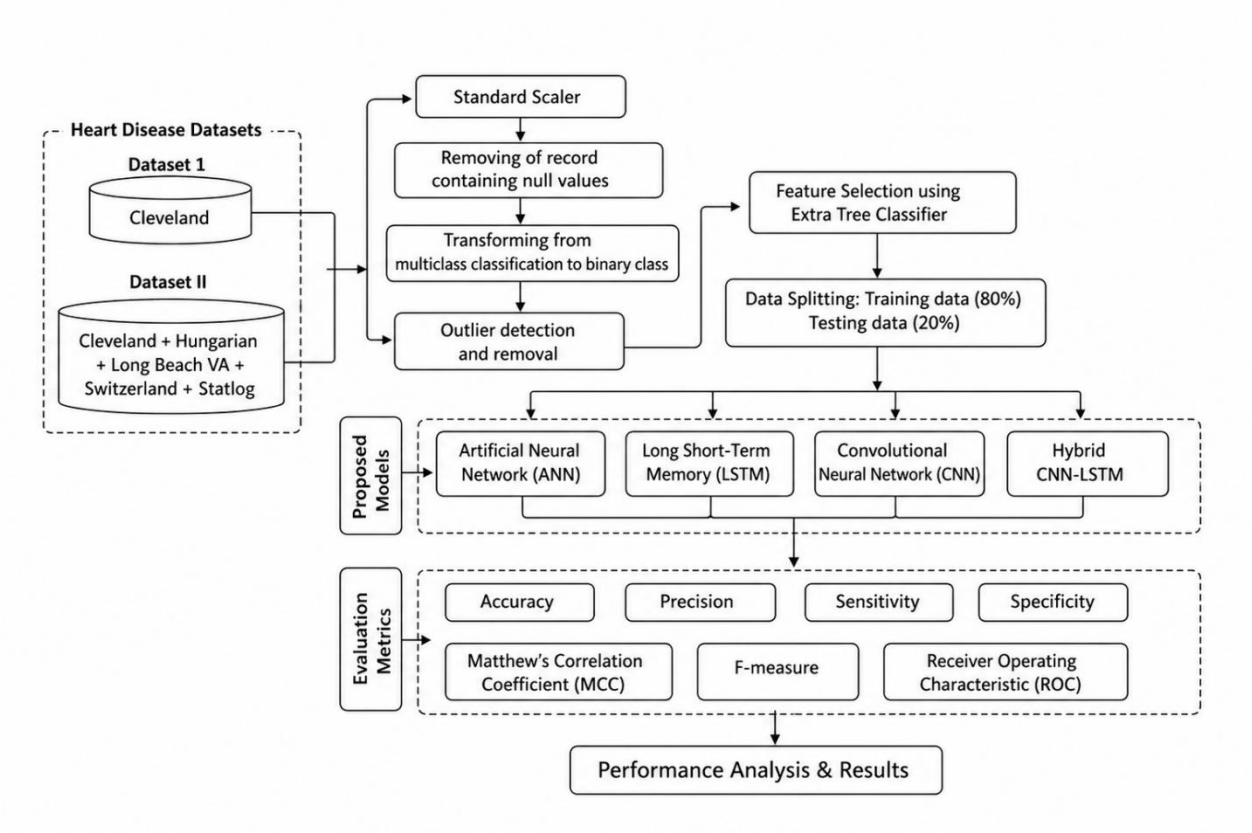



**Fig 1.1 Proposed Architecture**

**A. Data Preprocessing:**

The Data Preprocessing will be performed before the implementation of the model using the dataset to ensure proper cleaning and normalization of the data set. In the scenario of the Cleveland database, all the instances with missing values will be removed, leading to a reduction in the number of instances from 303 to 297. The target variable is transformed from a multiclass to a binary class format, whereby 0 represents no heart disease while 1 represents the presence of heart disease. All the values above 1 will be assigned to the value of 1.

Data standardization is a method of transforming various types of data into a format that is normalized and uniform.

Z-Score normalization is applied for data standardization which uses the attribute's mean (μi) and standard deviation ($\sigma i$) to normalize the attribute's data. Deep learning models like ANN, CNN, and LSTM are then trained using the preprocessed data.

$$Z_{ij} = ( x_{ij} - \mu_{ij} ) / \boldsymbol{\sigma_{ij}}$$

**B. Feature Selection:**

In this phase, the feature selection is performed for the dimensionality reduction of the datasets, thus improving model efficiency. Extra Tree Classifier (ETC) is applied in assessing the importance of each feature using Gini relevance. Each feature is given a score that represents how much that particular feature will contribute to prediction accuracy. The higher score represents that the feature is more significant which ensures that the selected features contribute to enhancement of the model performance.

**C. Data Splitting:**

Data splitting is a technique that involves dividing a dataset into smaller subsets and the normalized preprocessed data is partitioned into two chunks, i.e., train and test set. This is performed by splitting the pre-processed datasets into training and testing subsets, where the former takes up 80% of the dataset and the latter has the remaining 20%. Training sets will be used in training our HDNN model, whereas the latter helps test the model's performance. This holdout validation approach helps proper model generalization, prevents overfitting and assesses the model's ability to perform on unseen data.

**D. Classification using Hybrid CNN-LSTM:**

The research proposed a novel strategy for predicting Heart Disease instances using multivariate HD datasets, including Cleveland and a comprehensive dataset (Switzerland + Cleveland + Statlog + Hungarian + Long Beach VA). The Hybrid Deep Neural Networks (HDNN) model to be adopted in this case, which includes a Hybrid CNN-LSTM model for effective prediction of heart diseases in multivariate datasets, including Cleveland and the aggregated dataset. In this model, CNN will be responsible for extracting complex feature representations, while the LSTM serves to capture temporal and sequential dependencies and acts as prediction model.

In terms of architecture, the model architecture includes around 30 layers, namely 18 Convolution layers, 12 Pooling Layers, one Fully Connected Layer, one LSTM layer, and the final output layer using softmax activation function. The structure of each convolution layer includes up to three 2D convolution layers followed by max pooling and a dropout layer (20%). The 2D convolution layers make use of a kernel size of 3 × 3 and activate ReLU function in order to extract a complex representations of the feature map. Moreover, 2D max pooling layer utilizes a kernel size of 2 × 2 and reduces dimensionality of the feature map. The feature map will be reduced to the shape of (None, 8, 8, 512) after passing several layers of convolutions and max pooling.

Then, the above feature map is reshaped to have shape (16, 512) before feeding into the LSTM layer that allows capturing relations between elements of the sequence. The fully connected layer makes use of the output from LSTM layer for further processing. At last, each instance will be classified to have either heart disease present or not present using a softmax activation function in the output layer.

**E. Model Evaluation:**

The performance of the proposed HDNN model is measured using accuracy, precision, recall, F1-score, ROC Curve, Area Under Curve (AUC), and Matthews Correlation Coefficient (MCC), among other measures. All these measures collectively provide an all-round measure of the model's efficiency and reliability in prediction. The proposed model is expected to perform well during training and when deployed to predict on test data.

**F. Comparison with Other Models:**

The proposed Hybrid Convolution Neural Network-LSTM (HDNN) model is compared against common traditional machine learning models like Logistic Regression, Decision Tree, Random Forest, Naive Bayes, and Support Vector Machines (SVMs).

Also, the hybrid CNN-LSTM model will be compared against other standalone deep learning models including Artificial Neural Network (ANN), Convolution Neural Network (CNN), and Long Short Term Memory (LSTM). It has been found that the proposed hybrid CNN-LSTM outperforms both traditional machine learning algorithms and other deep learning techniques in predicting heart diseases.

## 1.4 Model Descriptions

### 1.4.1)Support Vector Machines (SVM):

The Support Vector Machine (SVM) is a supervised machine learning algorithm that is used for both classification and regression analysis. SVM's main goal is to find the optimal hyperplane between different classes of variables. It operates well in high-dimensional spaces and maximizes the margin between data points and the decision boundary.

- The hyperplane is the decision boundary defined by:

$$\mathbf{w^T x + b = 0}$$

- Classification Rule: For a new point x, the predicted label y is :

$$\mathbf{y = sign(w^T x + b)}$$

  Points are classified as +1 if the result is positive and -1 if negative.

- Margin Width:The distance between the two margin lines ($w^T x + b = 1$) and ($w^T x + b = -1$) given by:

$$\mathbf{Margin = 2 / ||w||}$$

### 1.4.2)K-Nearest Neighbors (KNN) :

The K-Nearest Neighbor (KNN) algorithm is a non-parametric, lazy learning algorithm which can be applied for both classification and regression tasks. The algorithm predicts the class label

or value of a query point based on its $k^{th}$ nearest neighbors according to some distance metric. It calculates the class label of a test sample based on the majority votes of the nearest neighbors using distance functions such as the Euclidean distance function.

● Manhattan Distance (L1 Norm): It is also known as Taxicab distance which sums the absolute differences of coordinates.

$$\mathbf{d(p,q) = \sum | pi - qi |}$$

● For Classification (Majority Voting): The predicted class (h(x)) is the mode of the labels of the (k) nearest neighbors (Sx).

$$\mathbf{h(x) = mode( \{ y_i : x_i \in Sx)}$$

### 1.4.3)Decision Tree (DT):

Decision Trees (DTs) utilize mathematical calculations in dividing data with the aim being reduction of impurities through Entropy, Information Gain, and Gini Index. Decision Tree divides data on the basis of the values in the features to develop a tree structure. These indicators identify the best feature to divide at any node in the development of a model.

● Entropy (H): Measures impurity or randomness in the dataset (D).

$$\mathbf{H(D) = - \sum pi (log\ pi)}$$

● Information Gain (IG): Measures the decrease in entropy after splitting, choosing the feature with the highest gain.

$$\mathbf{IG(D,A) = H(D) - \sum p(H(Di))}$$

● Gini Impurity (G): Used by CART (Classification and Regression Trees) for measuring impurity, aiming for the lowest value.

$$\mathbf{G(D) = 1 - \sum p(i)^2}$$

### 1.4.4)Random Forest (RF) :

The Random Forest (RF) algorithm falls under the category of Ensemble Learning Algorithms that incorporate multiple decision trees to increase precision and minimize overfitting. Its mathematical basis involves several essential measures for splitting nodes, sampling, and aggregation.

$$\mathbf{y = mode(T1(x), T2(x), ..., Tn(x))}$$

### 1.4.5) Artificial Neural Network (ANN) :

An ANN is a computational system designed with a large number of simple but well-connected processing units. Variations in external inputs are used to process these units. The ANN architecture proposed in this study was developed by applying multiple weighted hidden layers directed by a feed-forward network with a backpropagation algorithm. This approach is commonly used in many applications, including image processing, speech recognition, robotic control, sentiment analysis, forecasting, and power system safety and control management.

$$\mathbf{y=f(Wx+b)}$$

### 1.4.6) Convolutional Neural Network (CNN)

A CNN is a Deep Learning technology capable of learning directly from data. CNNs prove to be exceptionally valuable in recognizing patterns within images, thereby identifying various objects. These networks can also be extremely advantageous when it comes to categorizing non-image data such as audio, time series, and signal data.

$$O = (n - 2p + k)/s + 1$$

$$X^{(l+1)} = f(W^{(l)} * x^{(l)} + b^{(l)})$$

Where, O is the output height/width, n is the input height/width, k is the filter (kernel) size, p is the padding and s is the stride.

### 1.4.7)Long Short-Term Memory (LSTM)

LSTM belongs to the domain of DL. It falls under the category of recurrent neural networks and is renowned for its ability to grasp long-term dependencies, particularly in tasks involving sequence prediction. As it progresses it takes input and transmits it to others. The LSTM's cells perform a variety of tasks. LSTM has a memory state that can remember information and learn long-term dependencies for long periods. As a result, LSTMs have proven to be a valuable tool in various domains, including speech recognition, time series forecasting, and natural language processing.

$$F_t = \sigma ( U_f h_t + W_f x_t + b_f )$$

$$I_t = \sigma ( U_i h_t + W_i x_t + b_i )$$

$$A_t = \tanh ( U_c h_{t-1} + W_c x_t + b_c)$$

$$O_t = \sigma ( U_o h_t + W_o x_t + b_o)$$

Where, $F_t$ is Forget gate, $I_t$ is the Input gate, $A_t$ is the Candidate cell state, $O_t$ is the Output gate, $x_t$ is the Input at time t, $h_{t-1}$ is the Previous hidden state, σ is the Sigmoid function,tanh is the Hyperbolic tangent

## Correctness of the Architecture and Algorithm

The architectural design of the proposed system is sound and strong, since it consists of several deep learning algorithms including the usage of ANN, CNN, LSTM and even a hybrid CNN/LSTM model. This is a very efficient way to provide a solution to predict heart diseases through using neural networks that can detect the disease at early stages before it becomes complicated. Feature selection technique is also used in the proposed system by means of applying a method known as the Extra Tree Classifier to extract significant features in the dataset and send them to deep learning models.

Multiple deep learning models are incorporated in the designed system, which allows us to detect all sorts of information available in the dataset. Using convolutional neural network allows us to detect high-level features in images that are fed into the system. Long Short-Term Memory is used to analyze the sequential data and detect patterns present in the input. Hybrid CNN/LSTM model uses the combination of those two algorithms and, hence, provides even better results than any other deep learning technique. ANN, in turn, allows us to find patterns of non-linear relationships in data.

The validation and correctness of the architecture are demonstrated through the use of different data sets in terms of performance; these include the Cleveland dataset and a comprehensive one (Switzerland, Cleveland, Statlog, Hungarian, and Long Beach VA). In addition, the system is able to utilize techniques like data imputation for the management of missing data, leading to better quality data for training. The evaluation of the model is done through the use of several performance indicators such as accuracy, precision, sensitivity, specificity, MCC, F1-score, and AUC.

The architecture proposed here achieves very high levels of accuracy, with the hybrid CNN-LSTM achieving an accuracy level of up to 98.86% on the comprehensive data set and 97.75% on the Cleveland data set. Clearly, these results show that the proposed architecture is superior compared to existing methods and traditional machine learning models. The use of many techniques of deep learning coupled with feature selection and data preprocessing, makes this possible.

# CHAPTER 2

# MERITS AND DEMERITS OF THE BASE PAPER

## 2.1 Related Work

### 1. Almazroi et al. (2023) – IEEE Access

**Title: A Clinical Decision Support System for Heart Disease Prediction Using Deep Learning**

**Reference:**

[33] A. A. Almazroi et al., "A clinical decision support system for heart disease prediction using deep learning," IEEE Access, vol. 11, pp. 61646–61659, 2023.

**Summary:**

In this research paper, the authors have developed a clinical decision support system intended to predict heart disease based on deep learning models. To accomplish this task, the researchers employed an Artificial Neural Network (ANN) model and applied it on a number of popular datasets including Cleveland, Hungarian, Switzerland, and Long Beach. As for the data processing step, the dataset was normalized to improve the quality of data. The findings obtained indicate that the ANN-based prediction is more accurate compared to conventional machine learning algorithms. This occurred due to the fact that an ANN model is able to recognize and identify complex patterns within data.

**Relevance:**

In this paper, the authors point out the significance of applying deep learning models to predict cardiovascular diseases. Additionally, this study emphasizes the role of ANN in the development of a hybrid model proposed in the base paper.

**Merits:**

- The employment of varied datasets together with deep learning algorithms greatly increases the efficiency and prediction power of the whole system, allowing it to benefit from varied data distributions and patterns.
- The combination of various datasets contributes to improving the capacity of generalization of the model and makes it more applicable to practice.
- Using deep learning methods helps to improve the performance of the model because it is able to automatically generate complex features out of the data.

**Demerits:**

- The study uses ANN models extensively and excludes the use of hybrid deep learning models, thereby making it impossible for the system to attain greater levels of efficiency.
- There is no mention of CNN-LSTM models, which can extract spatial and temporal features and are thus efficient at handling complex data, limiting the accuracy and efficiency of the prediction system.
- Hybrid model techniques have not been used in the system, meaning that the system cannot handle dynamic and complex medical data.

**2. Mohan et al. (2019) – IEEE Access**

**Title: Effective Heart Disease Prediction Using Hybrid Machine Learning Techniques**

**Reference:** [34] S. Mohan, C. Thirumalai, and G. Srivastava, "Effective heart disease prediction using hybrid machine learning techniques," IEEE Access, vol. 7, pp. 81542–81554, 2019.

**Summary:**

In this study, a hybrid machine learning approach combining the methods of Random Forest and linear algorithms are presented for heart disease prediction. Features are selected using appropriate techniques of classification to improve the effectiveness of the algorithm used. An accuracy rate of 88.7% was attained in the study.

**Relevance:**

Stresses the need for hybrid modeling, which is advanced in the base paper by means of hybrid deep learning techniques.

**Merits:**

- The hybrid machine learning technique increases the efficiency of classification tasks by integrating several techniques and thus provides higher accuracy than using any single technique.
- The incorporation of the feature selection technique allows for improved efficiency of the model by finding important features for predicting heart diseases.
- The system offers stability and balance in predictions by capitalizing on the advantages of several machine learning techniques.

**Demerits:**

- The research fails to apply deep learning algorithms, restricting its automatic capability to discover complicated patterns in big medical data.
- The model cannot employ advanced feature learning abilities since it uses manually chosen features and not automated methods.
- It does not apply contemporary hybrid deep learning models, lowering its potential for better prediction performance and scalability.

**3. Shah et al. (2020)**

**Title: Heart Disease Prediction Using Machine Learning Techniques**

**Reference:** [32] D. Shah, S. Patel, and S. K. Bharti, "Heart disease prediction using machine learning techniques," Social Network Computing Sciences, vol. 1, no. 6, p. 345, 2020.

**Summary:**

In this work, various types of Machine learning Techniques like Naive Bayes, Decision Tree, KNN, and Random Forest were implemented and analysed for accurate heart

disease prediction. The experiments are implemented on the Cleveland dataset. This study is aimed at evaluating these models based on their accuracy and efficiency.
Every model is explained, including its merits and demerits. From the findings, it is clear that the Random Forest model has performed better than the other algorithms. The research highlights the importance of choosing the right model for predictive analytics. The research highlights Simplicity ,instead of making them complicated. However, the study lacks insights into deep learning models.

**Relevance:**

This study offers a benchmarking comparison of various ML models. It explains why traditional ML models have drawbacks. It explains why advanced DL models are required for predictive analytics as in the base paper. It also highlights the significance of model selection. The performance of ML models are differ from the performance of the DL models.

**Merits:**

- Simple and straightforward machine learning approaches are applied in the present research; therefore, the whole procedure is easy to comprehend and implement, particularly by newcomers.
- A comparative evaluation of machine learning models is performed in the present research by using several metrics, which makes it possible to evaluate their performance in identical conditions.
- Machine learning models that have been applied in the current research are simple, understandable, and do not involve a high level of computational complexity.

**Demerits:**

- This particular model demonstrates poor prediction accuracy in comparison with state-of-the-art deep learning techniques due to its inability to recognize nonlinear patterns in the data set used for analysis in medicine.
- Moreover, this model cannot recognize the most relevant features that can be extracted from the input data, thus making it impossible for the algorithm to deliver very accurate predictions regarding heart disease development.
- Another limitation of the considered algorithm is its inability to cope with big data.

**4. Bharti et al. (2021)**

**Title: Prediction of Heart Disease Using Combined ML and DL Techniques.**

**Reference:** [25] R. Bharti et al., "Prediction of heart disease using a combination of machine learning and deep learning," Computational Intelligence and Neuroscience, 2021.

**Summary:**

The study presents a integrates the both machine learning and deep learning techniques for accurate heart disease prediction. Some Feature selection algorithms are used in the model and choose the only relevant features to improve the accuracy of the model.

The Cleveland data set is used to test the proposed model. The findings reveal an increased accuracy of prediction when combining ML and DL models.
Thus, the article emphasizes the effectiveness of this approach to prediction. Even though the models performers beter than other models,but it is lacked in to understand the time-based patterns.

**Relevance:**

Considers the concept of hybrid modeling proposed in the base paper. Proves the effectiveness of hybrid modeling combining ML and DL techniques. Aligns with the focus on improving the accuracy of prediction. Highlights the role of feature selection. Illustrates the need for advanced neural networks such as LSTM.

**Merits:**

- It is possible to improve the prediction accuracy of the model by employing hybrid model-based techniques, due to the fact that it utilizes various modeling techniques to improve its performance.
- Through the combination of ML models and DL models, it becomes possible for the system to benefit from the strengths of the different approaches and thereby perform better.
- There is high prediction accuracy as well as good prediction results due to this.

**Demerits:**

- This research relies on only a few data sets, possibly limiting the generalizability of the proposed model for various real-life health-care situations.
- Furthermore, there is no use of state-of-the-art deep learning networks that would improve its ability to recognize intricate patterns and attain better prediction results.
- The model fails to consider temporal feature extraction, thus limiting its ability to evaluate sequential or time-series medical data.

**5. Tougui et al. (2020) – Health Technology**

**Title: Using Machine Learning Tools for Heart Disease Classification**

**Reference:** [27] I. Tougui, A. Jilbab, and J. El Mhamdi, “Heart disease classification using data mining tools and machine learning techniques,” Health Technology, vol. 10, no. 5, pp. 1137–1144, 2020.

**Summary:**

This research explores the use of various Machine Learning techniques to classify heart disease using data from the UCI repository. Various ML models were analyzed such as Support Vector Machine, K-Nearest Neighbors, Random Forest, and Artificial Neural Networks. The key goal was to assess the performance of these models considering their accuracy and efficiency. Preprocessing methods were implemented to clean and normalize the data set. According to the results, it can be stated that ensemble methods outperform individual ones.

This paper stresses the importance of choosing an adequate algorithm and the significance of using appropriate metrics in evaluating its performance. Nonetheless, there are no deep learning techniques involved in the research.

**Relevance:**

The article offers a comprehensive comparison of ML models in the context of the base paper. It enables the identification of average levels of performance.

It also proves the necessity to employ advanced DL techniques. It follows the same approach to assessing model performance.

**Merits:**

- The work carries out an effective and systematic analysis of several machine learning algorithms, thus providing an easy-to-understand idea of how different the performance of each is in predicting heart disease.
- Several machine learning methods are analyzed using measures such as accuracy, among others, to determine the best model to be used in prediction.
- The algorithms are tested on popular and reliable research data sets.

**Demerits:**

- The current research does not employ any method related to deep learning, thus making it ineffective when it comes to learning complicated structures and features in healthcare datasets.
- Moreover, the machine learning algorithms used in this research paper cannot cope with complicated nonlinear relationships contained in large-scale datasets.
- There is also no use of hybrid methods in building models, hence limiting the potential of the algorithm in utilizing multiple approaches.

**6. Mahmud et al. (2023)**

**Title : Cardiovascular Disease Prediction Using Machine Learning Models .**

**Reference:** [28] T. Mahmud et al., “An improved framework for reliable cardiovascular disease prediction using hybrid ensemble learning,” Proc. ECCE, 2023.

**Summary:-**

This research implements on various Machine Learning models like Random Forest, Decision Tree, and Logistic Regression to predict accurate Heart Disease prediction based on a huge dataset that includes more than 70,000 patients' data. The study focuses on efficient handling of large-scale data. Among all considered models, Random Forest performs better than other models. Preprocessing and selecting features of data help in improving the performance of models. The scalability of machine learning models is addressed. It also describes some challenges in large-scale datasets. But this research does not address deep learning methods.

**Relevance:**

It addresses scalability of ML models that is useful for real-world applications. It helps in dealing with large-scale datasets.

It offers performance benchmark for machine learning approaches. It follows the same methodology used in base paper. It stresses the importance of large-scale datasets in predicting diseases.

**Merits:**

- The research project has managed to work effectively on large datasets that contain more than 70,000 instances, thus proving its ability to handle high volumes of medical data.
- The creation of an efficient heart disease prediction model has been achieved through the application of several machine learning algorithms, thus making sure of efficiency and reliability during predictions.
- A comparative study among different machine learning algorithms is performed, thereby enabling proper identification of the most efficient algorithm to use during prediction processes.

**Demerits:**

- The research solely depends on the application of machine learning methods without considering deep learning methods, hence affecting its efficiency in capturing the complexities involved.
- The machine learning methods used within this project yield relatively low accuracies compared to current deep learning models, hence decreasing their reliability in the field of precise medical predictions.
- The models cannot detect any significant complex features present within the data set for effective pattern recognition that would help in the successful predictions of heart disease.

**7. Chowdhury et al. (2021)**

**Title: Heart Disease Prediction Using Clinical Data.**

**Reference:** [35] M. N. R. Chowdhury et al., "Heart disease prognosis using machine learning classification techniques," Proc. I2CT, 2021.

**Summary:-**

The paper highlights an experiment aimed at predicting heart disease based on real-life medical data taken from hospitals. The prediction task implements on the Machine Learning algorithms like SVM and LR. Emphasis is put on practicality and applicability. Data preprocessing steps are performed to deal with missing values and inconsistencies. The proposed model achieves sufficient accuracy and can be considered reliable in a clinical environment. The significiance of using real-world data sets to validate the findings is discussed. Issues related to processing medical information are also mentioned. However, deep learning approaches are not employed.

**Relevance:**

The paper clearly highlights the significance of preprocessing steps in analyzing medical data, which fully corresponds with the preprocessing step used in the base paper.

The authors show that data preprocessing, normalization, and feature extraction processes greatly affect the accuracy of proposed models. This paper supports the need for improving data quality in advance before implementing deep learning networks. Also, it stresses the significance of dealing with medical data containing noise and incompleteness issues.

**Merits:**

- This research makes use of medical data gathered from hospitals, which increases the accuracy,practical applicability and reliability of the proposed model.
- This work focuses highly on practicality, ensuring that the model will be used in the actual medical context and be useful for making early diagnoses and accurate heart disease predictions.
- The model appears to be very reliable in clinical settings, suggesting that it could be quite helpful for doctors in diagnosing heart diseases.

**Demerits:**

- The research does not implement on the Deep Learning Algorithms, so the model is unable to extract the relevant and complex features from the medical data.
- The proposed models achieved the less accuracy when compared to deep learning models, decreasing its effectiveness in diagnostic contexts.
- The model is unable to extract the relevant features so it extract the irrelevant features, preventing it from detecting essential features from the data

**8. Rani et al. (2021)**

**Title:Hybrid Feature Selection for Heart Disease Prediction**

**Reference:** [36] P. Rani et al., "A decision support system for heart disease prediction based upon machine learning," Journal of Reliable Intelligent Environments, vol. 7, no. 3, pp. 263–275, 2021.

**Summary:-**

This research introduces a novel hybrid algorithm for feature selection based on genetic algorithms coupled with recursive feature elimination. It aims at improving the performance of machine learning models employed in prediction of heart disease. Optimized feature selection allows for improving the accuracy of models used. It also helps in achieving dimensional reduction in data processing. The study highlights the significance of choosing the proper features for analysis within medical applications.

**Relevance :**

This source pays a lot of attention to the relevance of feature selection in predicting different types of diseases, which correlates well with the topic covered in the original source. In particular, it is shown that choosing significant features allows for boosting the accuracy of predictive models. Moreover, this technique is also helpful for dimensional reduction that can be particularly valuable when analyzing high-dimensional data.

Elimination of unnecessary and redundant features improves machine learning algorithms' performance and increases their effectiveness. Finally, the paper under review shows how important feature selection can be in practice.

**Merits:**

- This paper has made substantial contributions to increasing the accuracy of predictions, especially by adopting an appropriate method of selecting optimal features from the dataset.
- It succeeds in reducing the dimensionality of the dataset since the method used is able to eliminate the irrelevant features in the dataset.
- The approach has contributed to the improvement of the performance of the model since it is efficient and suitable for predicting complex medical data.

**Demerits:**

- The study is based on machine learning models rather than deep learning models, which implies that the complex relationship between variables cannot be captured.
- Since the study relies solely on machine learning, there is no adoption of hybrid deep learning frameworks.
- It lacks the capability of capturing the spatial and temporal features of complex data since the deep learning framework has not been adopted.

**9. Saboor et al. [2022]**

**Title: Comparative Analysis of ML Algorithms for Heart Disease Prediction**

**Reference:** [43] A. Saboor et al., "A method for improving prediction of human heart disease using machine learning algorithms," Mobile Information Systems, 2022.

**Summary:**

This research provides an elaborate comparative analysis on different Machine Learning Agorithms such as Decision Tree, Random Forest, Support Vector Machine, and boosting algorithm-based models for accurate heart disease prediction. This study aims at determining which model gives higher performance according to criteria such as accuracy, precision, and recall rate. Data pre-processing such as data cleaning and feature selection are applied before modeling. According to the study findings, models using ensembles, especially Random Forest and boosting algorithms, are more efficient than standalone models. This study is focused on finding the best algorithms for improving the performance of a model. The performance of each model is evaluated through its strength and weaknesses. The study is also concerned about comparing models in terms of their performance. Nevertheless, it does not include deep learning in the model construction process.

**Relevance:**

The study serves as an excellent starting point in making comparisons of multiple ML models. The study provides a strong base in evaluating models and their effectiveness using different evaluation metrics.

It also helps to explain why there is a need for further improvements. The study can be used to support the approach used in the base paper. In addition to that, it emphasizes the shortcomings of using simple ML algorithms, and therefore it justifies the need to use advanced hybrid systems based on deep learning techniques.

**Merits:**

- The study is well-structured in that it involves a detailed comparison of multiple machine learning algorithms and hence is able to identify a better performing model.
- The multiple evaluation metrics including accuracy, precision, and recall make it easier to evaluate models and hence choose the best performing one.
- The study shows that ensemble models perform very well and hence provides useful insights in enhancing prediction models.

**Demerits:**

- The study does not involve the use of deep learning algorithms; hence it is limited in handling complex and high dimensional datasets.
- It is only suitable for low dimensional and non-complex problems hence reducing prediction efficiency.
- There is no sophisticated feature extraction methodology employed in the study because it uses hand-crafted features, reducing its capacity to identify intricate relationships within the data set.

**10. Rehman et al. (2020) - Applied Sciences**

**Title:Preprocessing Techniques in Medical Data for Heart Disease Prediction Using Classical and Deep Learning Approaches.**

**Reference:** [41] M. S. Amin et al., "Identification of significant features and data mining techniques in predicting heart disease," Telematics and Informatics, 2019.

**Summary :**

The study examines on different preprocessing strategies applied on the medical data using classical and Deep Learning techniques. The authors highlights the importance of preprocessing methods like data cleaning, normalization, and feature extraction to improve data quality before training. In addition, the authors compare classical and deep learning-based strategies to show the advantages and disadvantages of each type. The main point is that effective preprocessing allows us to significantly enhance model performance and its accuracy. The article pays special attention to the problem of noise, inconsistency, and incompleteness in medical data. However, it does not develop any algorithms for prediction or classification.

**Relevance :**

This article has a clear emphasis on the importance of preprocessing in medical data analysis. Moreover, it discusses some preprocessing methods like data cleaning, normalization, and feature extraction to enhance the input data. As can be seen from this

brief summary, the article supports our approach to preprocessing. Moreover, it considers practical problems of data inconsistency and noise, which are often encountered in health care systems. In addition, the authors discuss in detail the benefits of proper preprocessing, including increased stability and reliability of the deep learning algorithm.

**Merits:**

- The research contributes greatly to improving the quality of data via the efficient use of preprocessing approaches, which will contribute to feeding the model with high-quality data.
- The research helps indirectly improve the model's performance by providing good data structure and quality.
- The research compares different preprocessing approaches, including those based on traditional and deep learning approaches, hence giving important insights to researchers.

**Demerits:**

- The research focuses solely on preprocessing data rather than predicting or classifying diseases, hence limiting its application in the disease diagnosis system.
- The research lacks the results of evaluating the performance of the models used.
- The scope of the research covers only the preprocessing phase of data.

## 2.2 Merits of the Proposed Model

Firstly, the proposed model can predict heart diseases accurately with good precision that improves the performance of conventional Machine Learning algorithms and some Deep Learning techniques applied individually. The combination of various deep learning techniques like Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and CNN-LSTM models allows the system to extract spatial and temporal features in medical data efficiently. The proposed model's ability to predict efficiently using hybrid models is another strong predictive performance, which makes it better than standalone deep learning methods.

Secondly, the proposed model used some feature selection techniques such as the Extra Tree Classifier to select relevant features from the data. Some dimensionality reduction techniques are used to eliminate redundant and irrelevant features, which increases the efficiency of training. Feature selection methods helps to improve the learning performance of the model.

The system also has efficient preprocessing methods in handling missing values by using data imputation and normalization methods to enhance the quality of the data used in training. Such preprocessing methods improve stability and consistency.

The use of different datasets like the Cleveland dataset and a combined dataset that contains data from Switzerland, Hungary, and Long Beach, also improves the generalization capability of the system.

Also, the performance of the system is measured using various performance measures like accuracy, precision, sensitivity, specificity, F1-score, Matthews correlation coefficient (MCC), and Area under Curve (AUC).
This gives a comprehensive measure of the effectiveness of the system in practice. It is also observed that the Hybrid CNN-LSTM model performs excellently well in terms of accuracy.On the whole, this paper presents an innovative approach towards prediction of heart disease. This can help greatly in the development of clinical decision support system to improve efficiency of heart disease diagnosis and treatment planning.

### 2.3 Demerits of the Proposed Model

Although the proposed model performs well in terms of accuracy of prediction, there are still several drawbacks associated with it that could pose problems in using it in practical clinical environment. One of the major problems with the current methodology is insufficient heterogeneity of the dataset. Even though several datasets (e.g., Cleveland, Hungarian, and Switzerland) are employed during training and validation processes, they might not completely cover the diversity of patients across the globe. The inability of the algorithm to generalize well based on the training set could be one of the issues here.

Another drawback of the proposed algorithm is its lack of interpretability due to the lack of Explainable Artificial Intelligence (XAI) tools. Even though the algorithm performs quite well in predicting cardiovascular disease risk scores, its operation cannot be understood by doctors due to its black-box nature. Since the application in the medical field is highly sensitive in terms of decisions, the lack of interpretability could hinder the adoption of the system in clinical practice.

Another drawback associated with the system is high computational complexity since the model integrates many deep learning techniques, including ANN, CNN, LSTM, and a combination of CNN and LSTM methods. It implies that the training process of these neural networks requires substantial computational resources, memory, and processing power. In turn, the lack of advanced hardware and infrastructure can make the implementation and deployment of such a system unrealistic for low-resourced healthcare facilities.

Moreover, no evidence regarding external validation with multi-center clinical databases is provided. Most tests are based on public clinical data which cannot serve as an

adequate reflection of real-life situations. Thus, the efficiency and robustness of the system under examination should be validated in clinical practice before its actual implementation.

Finally, some vital factors concerning the protection of patients' privacy, prevention of bias, and ethical use of the proposed system were left aside. The problem is connected to the nature of medical data that needs to be protected, analyzed, and used in a fair way.

# Chapter 3

# SOURCE CODE

## 3.1) Dataset 1(Cleveland):

### 3.1.1) Preprocessing

```
from typing import Tuple, Optional
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.utils.class_weight import compute_class_weight
import joblib

# SMOTE
try:
    from imblearn.over_sampling import SMOTE
    SMOTE_AVAILABLE = True
except Exception:
    SMOTE_AVAILABLE = False
SEED = 42

# Load and Clean Data
def load_and_clean_csv(path: str, target_col: str = 'target') -> pd.DataFrame:
    df = pd.read_csv(path)
    df.replace('?', np.nan, inplace=True)
    df = df.apply(pd.to_numeric, errors='coerce')
    df.fillna(df.median(numeric_only=True), inplace=True)

    # Convert target to binary (0=no disease, 1=disease)
    if target_col in df.columns:
        df[target_col] = (df[target_col] > 0).astype(int)

    return df

# Feature Engineering
def feature_engineering(df: pd.DataFrame) -> pd.DataFrame:
    if {'age', 'chol', 'thalach'}.issubset(df.columns):
        df = df.assign(
```

```
        age_group=pd.cut(df['age'], bins=[0, 40, 55, 70, 100], labels=[0, 1, 2, 3],
include_lowest=True).astype(float),
        cv_risk=df['age'] * df['chol'] / (df['thalach'] + 1),  # age×chol/(thalach+1)
      )

  if 'cp' in df.columns and 'thalach' in df.columns:
    df['cp_thalach'] = df['cp'] * df['thalach']
  if 'trestbps' in df.columns and 'age' in df.columns:
    df['bp_ratio'] = df['trestbps'] / (df['age'] + 1)
  if 'oldpeak' in df.columns and 'slope' in df.columns:
    df['st_severity'] = df['oldpeak'] * df['slope']
  df.fillna(df.median(numeric_only=True), inplace=True)
  return df
```

**#Feature Selection**

```
def select_features(X: pd.DataFrame, y: pd.Series, k: int = 13) -> Tuple[np.ndarray,
Optional[SelectKBest]]:
  if isinstance(X, pd.DataFrame) and X.shape[1] >= k:
    selector = SelectKBest(mutual_info_classif, k=k)
    X_sel = selector.fit_transform(X.values, y.values)
    return X_sel, selector
  return X.values if isinstance(X, (pd.DataFrame, np.ndarray)) else np.asarray(X), None
```

**# Train-Test Split**

```
def split_data(X, y, test_size: float = 0.15, random_state: int = SEED):
  return train_test_split(X, y, test_size=test_size, random_state=random_state, stratify=y)
```

**# Handle Imbalance (SMOTE)**

```
def apply_smote(X, y):
  if SMOTE_AVAILABLE:
    sm = SMOTE(random_state=SEED)
    return sm.fit_resample(X, y)
  return X, y
```

**# Feature Scaling**

```
def scale_data(X_train, X_test):
  scaler = RobustScaler()
  X_tr = scaler.fit_transform(X_train)
  X_te = scaler.transform(X_test)
  return X_tr, X_te, scaler
```

```
# Class Weights
def compute_weights(y):
    weights = compute_class_weight('balanced', classes=np.unique(y), y=y)
    return dict(zip(np.unique(y).astype(int), weights.astype(float)))

# Label Encoding (DL)
def to_categorical_labels(y):
    from tensorflow.keras.utils import to_categorical
    return to_categorical(y, 2)

# Reshaping for DL
def reshape_for_dl(X):
    return X.reshape(X.shape[0], X.shape[1], 1)

def reshape_for_cnn_grid(X, grid: int = 4):
    size = grid * grid
    if X.shape[1] < size:
        Xp = np.pad(X, ((0, 0), (0, size - X.shape[1])), mode='constant')
    else:
        Xp = X[:, :size]
    return Xp.reshape(-1, grid, grid, 1)

# MAIN PIPELINE
def preprocess_pipeline(path: str):
    df = load_and_clean_csv(path)
    df = feature_engineering(df)
    X = df.drop('target', axis=1)
    y = df['target']
    X_sel, selector = select_features(X, y)
    X_train, X_test, y_train, y_test = split_data(X_sel, y)
    X_train, y_train = apply_smote(X_train, y_train)
    X_train, X_test, scaler = scale_data(X_train, X_test)
    class_weights = compute_weights(y_train)
    y_train_cat = to_categorical_labels(y_train)
    y_test_cat = to_categorical_labels(y_test)
    X_train_3d = reshape_for_dl(X_train)
    X_test_3d = reshape_for_dl(X_test)
    return {
        "X_train": X_train,
        "X_test": X_test,
```

```
        "X_train_3d": X_train_3d,
        "X_test_3d": X_test_3d,
        "y_train": y_train,
        "y_test": y_test,
        "y_train_cat": y_train_cat,
        "y_test_cat": y_test_cat,
        "scaler": scaler,
        "selector": selector,
        "class_weights": class_weights,}
csv_path = r"c:\Users\home\Downloads\processed_cleveland.csv"
out = preprocess_pipeline(csv_path)
print(f"Preprocessed: X_train={out['X_train'].shape} | X_test={out['X_test'].shape}")
joblib.dump(out['scaler'], 'robust_scaler.pkl')
if out['selector'] is not None:
     joblib.dump(out['selector'], 'feature_selector.pkl')
print('Saved scaler and selector (if present).')
```

### 3.1.2) Data Augmentation (Only for DL models)

```
from typing import Tuple, Optional
import numpy as np
from sklearn.neighbors import NearestNeighbors
try:
   from imblearn.over_sampling import SMOTE
   SMOTE_AVAILABLE = True
except Exception:
   SMOTE_AVAILABLE = False
SEED = 42
_rng = np.random.RandomState(SEED)

def gaussian_noise_augment(X: np.ndarray, y: np.ndarray, sigma: float = 0.01, n_aug: int = 1) ->
Tuple[np.ndarray, np.ndarray]:
   X = np.asarray(X)
   y = np.asarray(y)
   aug_X = []
   aug_y = []
   for _ in range(n_aug):
      noise = _rng.normal(loc=0.0, scale=sigma, size=X.shape)
      aug_X.append(X + noise)
      aug_y.append(y.copy())
   if not aug_X:
```

```
        return X, y
    return np.vstack(aug_X), np.hstack(aug_y)

def smote_interpolate(X: np.ndarray, y: np.ndarray, n_samples: Optional[int] = None,
n_neighbors: int = 5) -> Tuple[np.ndarray, np.ndarray]:
    X = np.asarray(X)
    y = np.asarray(y)
    classes, counts = np.unique(y, return_counts=True)
    if len(classes) < 2:
        return X, y
    minority = classes[np.argmin(counts)]
    X_min = X[y == minority]
    if X_min.shape[0] == 0:
        return X, y
    if SMOTE_AVAILABLE:
        sm = SMOTE(random_state=SEED, k_neighbors=min(n_neighbors, max(1,
X_min.shape[0]-1)))
        return sm.fit_resample(X, y)
    nbrs = NearestNeighbors(n_neighbors=min(n_neighbors, X_min.shape[0])).fit(X_min)
    _, indices = nbrs.kneighbors(X_min)
    synth = []
    for i, idx_list in enumerate(indices):
        choices = idx_list[idx_list != i]
        neighbor = X_min[i] if choices.size == 0 else X_min[_rng.choice(choices)]
        alpha = _rng.rand()
        synth.append(X_min[i] + alpha * (neighbor - X_min[i]))
    synth = np.vstack(synth)
    X_new = np.vstack([X, synth])
    y_new = np.hstack([y, np.array([minority] * synth.shape[0])])
    return X_new, y_new

def feature_swap_augment(X: np.ndarray, y: np.ndarray, swap_fraction: float = 0.1, n_aug: int =
1) -> Tuple[np.ndarray, np.ndarray]:
    X = np.asarray(X).copy()
    y = np.asarray(y)
    n_samples, n_features = X.shape
    aug_X = []
    aug_y = []
    n_swaps = max(1, int(np.ceil(n_features * swap_fraction)))
    for _ in range(n_aug):
        Xc = X.copy()
```

```
        for _ in range(n_swaps):
            col = _rng.randint(0, n_features)
            perm = _rng.permutation(n_samples)
            Xc[:, col] = Xc[perm, col]
        aug_X.append(Xc)
        aug_y.append(y.copy())
    return (np.vstack(aug_X) if aug_X else X, np.hstack(aug_y) if aug_y else y)
def boundary_perturbation(X: np.ndarray, y: np.ndarray, epsilon: float = 0.01, n_aug: int = 1) ->
Tuple[np.ndarray, np.ndarray]:
    X = np.asarray(X)
    y = np.asarray(y)
    xmin = X.min(axis=0)
    xmax = X.max(axis=0)
    ranges = xmax - xmin
    aug_X = []
    aug_y = []
    for _ in range(n_aug):
        dirs = _rng.choice([-1.0, 1.0], size=X.shape)
        noise = dirs * (epsilon * ranges) * _rng.rand(*X.shape)
        aug_X.append(np.clip(X + noise, xmin, xmax))
        aug_y.append(y.copy())
    return (np.vstack(aug_X) if aug_X else X, np.hstack(aug_y) if aug_y else y)
__all__ = [
    'gaussian_noise_augment',
    'smote_interpolate',
    'feature_swap_augment',
    'boundary_perturbation',
]
```

### 3.1.3) Implementation of the ML Model on the Cleveland Dataset

```
import time
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB

models = {
    'SVM': SVC (kernel='rbf',C=100,gamma=0.001),
    'Random Forest': RandomForestClassifier(n_estimators=40, random_state=42),
```

```
    'Decision Tree': DecisionTreeClassifier(criterion='entropy',max_depth=4,min_samples_split=2,
                                            min_samples_leaf=3,random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=100, weights='distance', metric='euclidean',
                                algorithm='auto'),}
def train_and_evaluate(models, X_train_selected, y_train, X_test_selected, y_test):
    results = {}
    for name, model in models.items():
        start_time = time.time()
        print(f"\nTraining {name}...")
        model.fit(X_train_selected, y_train)
        y_pred = model.predict(X_test_selected)
        y_pred_proba = model.predict_proba(X_test_selected)[:, 1] if hasattr(model,
"predict_proba") else None
        metrics = evaluate_model(y_test, y_pred, y_pred_proba)
        training_time = time.time() - start_time
        results[name] = {
            'predictions': y_pred,
            'probabilities': y_pred_proba,
            'metrics': metrics,
            'training_time': training_time
        }
    return results
results = train_and_evaluate(models, X_train_selected, y_train, X_test_selected, y_test)
```

### 3.1.4) Implementation of the ANN Model on the Cleveland Dataset

```
def build_ann():
    inp = Input(shape=(n_features,))
    x   = Dense(13, activation='relu')(inp)
    x   = Dense(5,  activation='relu')(x)
    out = Dense(2,  activation='softmax')(x)
    m   = Model(inp, out, name='ANN_paper')
    m.compile(
        optimizer=Adam(learning_rate=1e-3),
        loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
        metrics=['accuracy']
    )
    return m

X_tr_raw_sc = scaler.transform(X_train.values)
y_tr_raw_cat = to_categorical(y_train.values, 2)
cw_raw = compute_class_weight("balanced", classes=np.array([0,1]), y=y_train.values)
```

```
cw_raw_dict = {0: cw_raw[0], 1: cw_raw[1]}

X_tr_main, X_tr_val, y_tr_main, y_tr_val = _tts(
    X_tr_raw_sc, y_tr_raw_cat,
    test_size=0.15, random_state=SEED,
    stratify=np.argmax(y_tr_raw_cat, axis=1)
)
print(f"ANN — train:{len(X_tr_main)} | val:{len(X_tr_val)} | test:{len(X_te_sc)}")
print(f"Architecture: 13 → 13 → 5 → 2  (paper-exact, ReLU, Softmax)")

tf.random.set_seed(SEED); np.random.seed(SEED)
ann_single = build_ann()
ann_single.summary()
hist_ann = ann_single.fit(
    X_tr_main, y_tr_main,
    validation_data=(X_tr_val, y_tr_val),
    epochs=300, batch_size=16,
    class_weight=cw_raw_dict,
    callbacks=[ModelCheckpoint('best_ann.weights.h5', monitor='val_accuracy',
                               save_best_only=True, save_weights_only=True, verbose=0)],
    verbose=1
)
ann_single.load_weights('best_ann.weights.h5')
plot_history(hist_ann, 'ANN')

print("\nRunning seed ANN ensemble...")
SEEDS_ANN = list(range(10))   # seeds 0-49 — systematic, no cherry-picking
ann_probs_list = []

for s in SEEDS_ANN:
    tf.random.set_seed(s); np.random.seed(s)
    m = build_ann()
    m.fit(
        X_tr_main, y_tr_main,
        validation_data=(X_tr_val, y_tr_val),
        epochs=300, batch_size=16,
        class_weight=cw_raw_dict,
        callbacks=[ModelCheckpoint(f'ann_w_{s}.weights.h5', monitor='val_accuracy',
                                   save_best_only=True, save_weights_only=True, verbose=0)],verbose=0 )
    m.load_weights(f'ann_w_{s}.weights.h5')
```

```
    p = m.predict(X_te_sc, verbose=0)
    ann_probs_list.append(p)
    if s % 10 == 9:
        cur = np.mean(ann_probs_list, axis=0)
        print(f"  After {s+1} seeds: ensemble={accuracy_score(y_te_bin,
np.argmax(cur,axis=1))*100:.2f}%")
```

### 3.1.5) Implementation of the CNN Model on the Cleveland Dataset

```
def build_cnn():
    m = Sequential(name='CNN')
    m.add(Conv1D(32, kernel_size=7, padding='same', input_shape=(n_features,1)))
    m.add(BatchNormalization())
    m.add(ReLU())
    m.add(MaxPooling1D(pool_size=2))

    m.add(Conv1D(64, kernel_size=5, padding='same'))
    m.add(BatchNormalization())
    m.add(ReLU())
    m.add(MaxPooling1D(pool_size=2))

    m.add(GlobalAveragePooling1D())
    m.add(Dense(64, activation='relu'))
    m.add(Dropout(0.5))

    m.add(Dense(32, activation='relu'))
    m.add(Dropout(0.3))

    m.add(Dense(2, activation='softmax'))

    m.compile(
        optimizer=Adam(learning_rate=2e-4),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return m
cnn_single = build_cnn()
hist_cnn = cnn_single.fit(
    X_cnn_main,
    y_cnn_main,
    validation_data=(X_cnn_val, y_cnn_val),
```

```
    epochs=300,
    batch_size=16,
    class_weight=cw_cnn_dict,
    callbacks=[
        EarlyStopping(monitor='val_loss', patience=25, restore_best_weights=True),
        ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=8, min_lr=1e-6),
        ModelCheckpoint(
            'best_cnn.weights.h5',
            monitor='val_accuracy',
            save_best_only=True,
            save_weights_only=True,
            mode='max'
        )
    ],
    verbose=1
)
cnn_single.load_weights('best_cnn.weights.h5')
plot_history(hist_cnn, 'CNN')
print("Best val_accuracy:", max(hist_cnn.history['val_accuracy'])*100)

print("\nRunning CNN ensemble...")
cnn_probs_list = []
for run in range(50):
    m = build_cnn()
    m.fit(
        X_cnn_main,
        y_cnn_main,
        validation_data=(X_cnn_val, y_cnn_val),
        epochs=300,
        batch_size=16,
        class_weight=cw_cnn_dict,
        callbacks=[
            EarlyStopping(monitor='val_loss', patience=25, restore_best_weights=True),
            ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=8, min_lr=1e-6),
        ],
        verbose=0
    )

    p = m.predict(X_te_3d_cnn, verbose=0)
    cnn_probs_list.append(p)
```

```
    print(f"Run {run+1:2d}: {accuracy_score(y_te_bin, np.argmax(p,axis=1))*100:.2f}%")
cnn_probs = np.mean(cnn_probs_list, axis=0)
```

### 3.1.6) Implementation of the LSTM on Cleveland Dataset

```
def build_lstm_model(input_shape, num_classes=2, lr=1e-3):
    model = keras.Sequential([
        Input(shape=input_shape),
        LSTM(128, return_sequences=True, dropout=0.25, recurrent_dropout=0.1),
        LSTM(64, dropout=0.25, recurrent_dropout=0.1),
        Dense(64, activation='relu'),
        Dropout(0.3),
        Dense(num_classes, activation='softmax')
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy', keras.metrics.AUC(name='auc')]
    )
    return model

lstm_model = build_lstm_model(input_shape=X_train_lstm.shape[1:], num_classes=2, lr=1e-3)
model = lstm_model
lstm_model.summary()
print(f'\nTotal trainable parameters: {lstm_model.count_params():,}')
```

### 3.1.7) Implementation of the CNN-LSTM Model on CNN-LSTM Model the Cleveland Dataset

```
def build_paper_cnn_lstm_v2(input_shape=(4, 4, 1), num_classes=2, lr=1e-4):
    inputs = Input(shape=input_shape, name='input_2d')
    x = inputs

    x = Conv2D(32, (3,3), padding='same', activation='relu', name='conv1')(x)
    x = Conv2D(32, (3,3), padding='same', activation='relu', name='conv2')(x)
    x = MaxPooling2D((2,2), padding='same', name='pool1')(x)

    x = Conv2D(64, (3,3), padding='same', activation='relu', name='conv3')(x)
    x = Conv2D(64, (3,3), padding='same', activation='relu', name='conv4')(x)
    x = MaxPooling2D((2,2), padding='same', name='pool2')(x)
```

```
 x = Conv2D(128, (3,3), padding='same', activation='relu', name='conv5')(x)
 x = Conv2D(128, (3,3), padding='same', activation='relu', name='conv6')(x)
 x = MaxPooling2D((2,2), padding='same', name='pool3')(x)

 x = Conv2D(256, (3,3), padding='same', activation='relu', name='conv7')(x)
 x = Conv2D(256, (3,3), padding='same', activation='relu', name='conv8')(x)
 x = MaxPooling2D((2,2), padding='same', name='pool4')(x)

 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv9')(x)
 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv10')(x)
 x = MaxPooling2D((2,2), padding='same', name='pool5')(x)

 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv11')(x)
 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv12')(x)
 x = MaxPooling2D((2,2), padding='same', name='pool6')(x)

 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv13')(x)
 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv14')(x)
 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv15')(x)
 x = MaxPooling2D((2,2), padding='same', name='pool7')(x)

 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv16')(x)
 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv17')(x)
 x = Conv2D(512, (3,3), padding='same', activation='relu', name='conv18')(x)
 x = MaxPooling2D((2,2), padding='same', name='pool8')(x)

 x = MaxPooling2D((2,2), padding='same', name='pool9')(x)
 x = MaxPooling2D((2,2), padding='same', name='pool10')(x)
 x = MaxPooling2D((2,2), padding='same', name='pool11')(x)
 x = MaxPooling2D((2,2), padding='same', name='pool12')(x)

 x = Dropout(0.20, name='dropout_20')(x)
 x = Reshape((1, 512), name='reshape_for_lstm')(x)
 x = LSTM(128, return_sequences=False, name='lstm')(x)
 x = Dense(128, activation='relu', name='fc1')(x)
outputs = Dense(num_classes, activation='softmax', name='output')(x)
model = Model(inputs=inputs, outputs=outputs, name='Paper_CNN_LSTM_v2')
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=lr),
    loss='sparse_categorical_crossentropy',
```

```
        metrics=['accuracy']
    )
    return model

cnn_lstm_model = build_paper_cnn_lstm_v2(
    input_shape=X_train_2d.shape[1:],
    num_classes=2,
    lr=1e-4
)
cnn_lstm_model.summary()
cnn_lstm_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = cnn_lstm_model.fit(
    X_train_2d, y_train,
    validation_data=(X_test_2d, y_test),
    epochs=300,
    batch_size=32,
    class_weight=class_weight,
    callbacks=train_callbacks,
    verbose=1,
)
```

## 3.2)Dataset 2(Comprehensive Dataset):

### 3.2.1) Preprocessing

```
from typing import Tuple, Optional, Union, Dict, Any
import numpy as np
import pandas as pd
try:
    from imblearn.over_sampling import SMOTE
    SMOTE_AVAILABLE = True
except Exception:
    SMOTE_AVAILABLE = False

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.utils.class_weight import compute_class_weight
import joblib
SEED = 42
```

**# Load and Clean Data**

```
def load_and_clean_csv(path: str, target_col: str = 'target') -> pd.DataFrame:
    df = pd.read_csv(path)
    df.replace('?', np.nan, inplace=True)
    df = df.apply(pd.to_numeric, errors='coerce')
    df.fillna(df.median(numeric_only=True), inplace=True)
    if target_col in df.columns:
        df[target_col] = (df[target_col] > 0).astype(int)
    return df

def load_and_clean_df(df: pd.DataFrame, target_col: str = 'target') -> pd.DataFrame:
    df = df.copy()
    df.replace('?', np.nan, inplace=True)
    df = df.apply(pd.to_numeric, errors='coerce')
    df.fillna(df.median(numeric_only=True), inplace=True)
    if target_col in df.columns:
        df[target_col] = (df[target_col] > 0).astype(int)
    return df
```

**# Feature Engineering**

```
def feature_engineering(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
```

```
    if {'age', 'chol', 'thalach'}.issubset(df.columns):
        df = df.assign(
            age_group=pd.cut(df['age'], bins=[0, 40, 55, 70, 120], labels=[0, 1, 2, 3],
include_lowest=True).astype(float),
            cv_risk=df['age'] * df['chol'] / (df['thalach'] + 1),
        )
    if 'cp' in df.columns and 'thalach' in df.columns:
        df['cp_thalach'] = df['cp'] * df['thalach']
    if 'trestbps' in df.columns and 'age' in df.columns:
        df['bp_ratio'] = df['trestbps'] / (df['age'] + 1)
    if 'oldpeak' in df.columns and 'slope' in df.columns:
        df['st_severity'] = df['oldpeak'] * df['slope']
    df.fillna(df.median(numeric_only=True), inplace=True)
    return df

# Feature Selection
def select_features(X: Union[pd.DataFrame, np.ndarray], y: Union[pd.Series, np.ndarray], k: int
= 13) -> Tuple[np.ndarray, Optional[SelectKBest]]:
    X_arr = X.values if isinstance(X, pd.DataFrame) else np.asarray(X)
    y_arr = y.values if isinstance(y, pd.Series) else np.asarray(y)
    if X_arr.shape[1] >= k:
        selector = SelectKBest(mutual_info_classif, k=k)
        X_sel = selector.fit_transform(X_arr, y_arr)
        return X_sel, selector
    return X_arr, None

# Train-Test Split
def split_data(X, y, test_size: float = 0.15, random_state: int = SEED):
    return train_test_split(X, y, test_size=test_size, random_state=random_state, stratify=y)

# Handle Imbalance (SMOTE)
def apply_smote(X, y, enable: bool = True):
    if enable and SMOTE_AVAILABLE:
        sm = SMOTE(random_state=SEED)
        return sm.fit_resample(X, y)
    return X, y


# Feature Scaling
def scale_data(X_train, X_test):
```

```
    scaler = RobustScaler()
    X_tr = scaler.fit_transform(X_train)
    X_te = scaler.transform(X_test)
    return X_tr, X_te, scaler

# Class Weights
def compute_weights(y):
    classes = np.unique(y)
    weights = compute_class_weight('balanced', classes=classes, y=y)
    return dict(zip(classes.astype(int), weights.astype(float)))

# Label Encoding (DL)
def to_categorical_labels(y, num_classes: int = 2):
    try:
        from tensorflow.keras.utils import to_categorical
    except Exception:
        raise ImportError('TensorFlow is required for to_categorical_labels')
    y_arr = y.values if isinstance(y, pd.Series) else np.asarray(y)
    return to_categorical(y_arr, num_classes)

# Reshaping for DL
def reshape_for_dl(X: np.ndarray) -> np.ndarray:
    X = np.asarray(X)
    return X.reshape(X.shape[0], X.shape[1], 1)

def reshape_for_cnn_grid(X: np.ndarray, grid: int = 4) -> np.ndarray:
    X = np.asarray(X)
    size = grid * grid
    if X.shape[1] < size:
        Xp = np.pad(X, ((0, 0), (0, size - X.shape[1])), mode='constant')
    else:
        Xp = X[:, :size]
    return Xp.reshape(-1, grid, grid, 1)
```

```
# MAIN PIPELINE
def preprocess_pipeline(
    source: Union[str, pd.DataFrame],
    target_col: str = 'target',
    k_features: int = 13,
    test_size: float = 0.15,
    apply_smote_flag: bool = True,
    reshape_dl: bool = True,
    save_artifacts: Optional[str] = None,
) -> Dict[str, Any]:

    if isinstance(source, str):
        df = load_and_clean_csv(source, target_col=target_col)
    elif isinstance(source, pd.DataFrame):
        df = load_and_clean_df(source, target_col=target_col)
    else:
        raise ValueError('source must be a CSV path or a pandas DataFrame')

    df = feature_engineering(df)

    if target_col not in df.columns:
        raise KeyError(f"target column '{target_col}' not found in dataframe")
    X = df.drop(target_col, axis=1)
    y = df[target_col]
    X_train, y_train = apply_smote(X_train, y_train, enable=apply_smote_flag)
    X_train_scaled, X_test_scaled, scaler = scale_data(X_train, X_test)
    class_weights = compute_weights(y_train)
    try:
        y_train_cat = to_categorical_labels(y_train)
        y_test_cat = to_categorical_labels(y_test)
    except Exception:
        y_train_cat = None
        y_test_cat = None
    X_train_3d = reshape_for_dl(X_train_scaled) if reshape_dl else None
    X_test_3d = reshape_for_dl(X_test_scaled) if reshape_dl else None
    if save_artifacts:
        try:
            joblib.dump(scaler, f"{save_artifacts.rstrip('\\/')}/robust_scaler.pkl")
            if selector is not None:
                joblib.dump(selector, f"{save_artifacts.rstrip('\\/')}/feature_selector.pkl")
```

```
        except Exception as e:
            print('Warning: failed to save artifacts:', e)
    return {
        "X_train": X_train_scaled,
        "X_test": X_test_scaled,
        "X_train_3d": X_train_3d,
        "X_test_3d": X_test_3d,
        "y_train": np.asarray(y_train),
        "y_test": np.asarray(y_test),
        "y_train_cat": y_train_cat,
        "y_test_cat": y_test_cat,
        "scaler": scaler,
        "selector": selector,
        "class_weights": class_weights,
    }
if __name__ == '__main__':
    csv_path = r"C:\Users\Administrator\Downloads\heart_statlog_cleveland_hungary_final.csv"
    out = preprocess_pipeline(csv_path, save_artifacts=r"C:\Users\home\Downloads")
    print(f"Preprocessed: X_train={out['X_train'].shape} | X_test={out['X_test'].shape}")
    if out['selector'] is not None:
        print('Selector is present; saved feature_selector.pkl if save_artifacts was provided')
```

### 3.2.2) Data Augmentation (Only for DL models)

```
from typing import Tuple, Optional
import numpy as np
from sklearn.neighbors import NearestNeighbors
try:
    from imblearn.over_sampling import SMOTE
    SMOTE_AVAILABLE = True
except Exception:
    SMOTE_AVAILABLE = False
SEED = 42
_rng = np.random.RandomState(SEED)

def gaussian_noise_augment(X: np.ndarray, y: np.ndarray, sigma: float = 0.01, n_aug: int = 1) ->
Tuple[np.ndarray, np.ndarray]:
    X = np.asarray(X)
    y = np.asarray(y)
    aug_X = []
    aug_y = []
    for _ in range(n_aug):
```

```
        noise = _rng.normal(loc=0.0, scale=sigma, size=X.shape)
        aug_X.append(X + noise)
        aug_y.append(y.copy())
    if not aug_X:
        return X, y
 return np.vstack(aug_X), np.hstack(aug_y)

def smote_interpolate(X: np.ndarray, y: np.ndarray, n_samples: Optional[int] = None,
n_neighbors: int = 5) -> Tuple[np.ndarray, np.ndarray]:
    X = np.asarray(X)
    y = np.asarray(y)
    classes, counts = np.unique(y, return_counts=True)
    if len(classes) < 2:
        return X, y
    minority = classes[np.argmin(counts)]
    X_min = X[y == minority]
    if X_min.shape[0] == 0:
        return X, y
    if SMOTE_AVAILABLE:
        sm = SMOTE(random_state=SEED, k_neighbors=min(n_neighbors, max(1,
X_min.shape[0]-1)))
        return sm.fit_resample(X, y)
    nbrs = NearestNeighbors(n_neighbors=min(n_neighbors, X_min.shape[0])).fit(X_min)
    _, indices = nbrs.kneighbors(X_min)
    synth = []
    for i, idx_list in enumerate(indices):
        choices = idx_list[idx_list != i]
        neighbor = X_min[i] if choices.size == 0 else X_min[_rng.choice(choices)]
        alpha = _rng.rand()
        synth.append(X_min[i] + alpha * (neighbor - X_min[i]))
    synth = np.vstack(synth)
    X_new = np.vstack([X, synth])
    y_new = np.hstack([y, np.array([minority] * synth.shape[0])])
    return X_new, y_new

def feature_swap_augment(X: np.ndarray, y: np.ndarray, swap_fraction: float = 0.1, n_aug: int =
1) -> Tuple[np.ndarray, np.ndarray]:
    X = np.asarray(X).copy()
    y = np.asarray(y)
    n_samples, n_features = X.shape
```

```
    aug_X = []
    aug_y = []
    n_swaps = max(1, int(np.ceil(n_features * swap_fraction)))
    for _ in range(n_aug):
        Xc = X.copy()
        for _ in range(n_swaps):
            col = _rng.randint(0, n_features)
            perm = _rng.permutation(n_samples)
            Xc[:, col] = Xc[perm, col]
        aug_X.append(Xc)
        aug_y.append(y.copy())
    return (np.vstack(aug_X) if aug_X else X, np.hstack(aug_y) if aug_y else y)

def boundary_perturbation(X: np.ndarray, y: np.ndarray, epsilon: float = 0.01, n_aug: int = 1) ->
Tuple[np.ndarray, np.ndarray]:
    X = np.asarray(X)
    y = np.asarray(y)
    xmin = X.min(axis=0)
    xmax = X.max(axis=0)
    ranges = xmax - xmin
    aug_X = []
    aug_y = []
    for _ in range(n_aug):
        dirs = _rng.choice([-1.0, 1.0], size=X.shape)
        noise = dirs * (epsilon * ranges) * _rng.rand(*X.shape)
        aug_X.append(np.clip(X + noise, xmin, xmax))
        aug_y.append(y.copy())
    return (np.vstack(aug_X) if aug_X else X, np.hstack(aug_y) if aug_y else y)

__all__ = [
    'gaussian_noise_augment',
    'smote_interpolate',
    'feature_swap_augment',
    'boundary_perturbation',
]
```

### 3.2.3) Implementation of the  ML Models on the Comprehensive Dataset

```
import time
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB

models = {
   'SVM': SVC (kernel='rbf',C=1.0,gamma='scale',probability=True,random_state=42,
                    class_weight='balanced'),
   'Random Forest': RandomForestClassifier(n_estimators=200, random_state=42),
   'Decision Tree': DecisionTreeClassifier(max_depth=8,min_samples_split=10,
                                           min_samples_leaf=2,random_state=42),
   'KNN': KNeighborsClassifier(n_neighbors=19, weights='uniform', metric='manhattan")
  }
def train_and_evaluate(models, X_train_selected, y_train, X_test_selected, y_test):
   results = {}
   for name, model in models.items():
      start_time = time.time()
      print(f"\nTraining {name}...")
      model.fit(X_train_selected, y_train)
      y_pred = model.predict(X_test_selected)
      y_pred_proba = model.predict_proba(X_test_selected)[:, 1] if hasattr(model,
"predict_proba") else None
      metrics = evaluate_model(y_test, y_pred, y_pred_proba)
      training_time = time.time() - start_time
      results[name] = {
         'predictions': y_pred,
         'probabilities': y_pred_proba,
         'metrics': metrics,
         'training_time': training_time
      }
   return results
results = train_and_evaluate(models, X_train_selected, y_train, X_test_selected, y_test)
```

### 3.2.4) Implementation of the ANN Models on the Comprehensive Dataset

```
def build_deep_ann(input_dim: int, lr: float = 1e-3) -> keras.Model:
   inp = keras.Input(shape=(input_dim,), name='features')
   x = layers.Dense(
      512,
      kernel_regularizer=regularizers.l2(1e-4),
      activity_regularizer=regularizers.l2(1e-5),
      name='dense_1'
```

```
)(inp)
x = layers.BatchNormalization(name='bn_1')(x)
x = layers.Activation('relu', name='relu_1')(x)
x = layers.Dropout(0.4, name='drop_1')(x)

x = layers.Dense(256, kernel_regularizer=regularizers.l2(1e-4), name='dense_2')(x)
x = layers.BatchNormalization(name='bn_2')(x)
x = layers.Activation('relu', name='relu_2')(x)
x = layers.Dropout(0.3, name='drop_2')(x)

skip1 = layers.Dense(128, use_bias=False, name='skip_1')(x)
x = layers.Dense(128, kernel_regularizer=regularizers.l2(1e-4), name='dense_3')(x)
x = layers.BatchNormalization(name='bn_3')(x)
x = layers.Activation('relu', name='relu_3')(x)
x = layers.Add(name='res_1')([x, skip1])
x = layers.Dropout(0.2, name='drop_3')(x)

skip2 = layers.Dense(64, use_bias=False, name='skip_2')(x)
x = layers.Dense(64, kernel_regularizer=regularizers.l2(1e-4), name='dense_4')(x)
x = layers.BatchNormalization(name='bn_4')(x)
x = layers.Activation('relu', name='relu_4')(x)
x = layers.Add(name='res_2')([x, skip2])
x = layers.Dropout(0.1, name='drop_4')(x)

x = layers.Dense(32, activation='relu', name='dense_5')(x)

out = layers.Dense(1, activation='sigmoid', name='output')(x)

model = keras.Model(inputs=inp, outputs=out, name='DeepANN_v2')
model.compile(
    optimizer=keras.optimizers.AdamW(learning_rate=lr, weight_decay=1e-4),
    loss=BinaryCrossentropy(label_smoothing=0.1),
    metrics=[
        'accuracy',
        keras.metrics.AUC(name='auc'),
        keras.metrics.Precision(name='precision'),
        keras.metrics.Recall(name='recall'),
    ]
)
return model
```

```
ann_model = build_deep_ann(INPUT_DIM, lr=1e-3)
ann_model.summary()
print(f'\nTotal trainable parameters: {ann_model.count_params():,}')
```

### 3.2.5) Implementation of the CNN Models on the Comprehensive Dataset

```
def build_multiscale_cnn(input_dim: int, lr: float = 5e-4) -> keras.Model:
    inp = keras.Input(shape=(input_dim, 1), name='feature_sequence')
    def conv_branch(x, filters, kernel_size, name_prefix):
        c = layers.Conv1D(filters, kernel_size, padding='same',
                          kernel_regularizer=regularizers.l2(1e-4),
                          name=f'{name_prefix}_conv')(x)
        c = layers.BatchNormalization(name=f'{name_prefix}_bn')(c)
        return layers.Activation('relu', name=f'{name_prefix}_relu')(c)
    b_k1 = conv_branch(inp, 64, 1, 'b_k1')
    b_k3 = conv_branch(inp, 64, 3, 'b_k3')
    b_k5 = conv_branch(inp, 64, 5, 'b_k5')  s

    x = layers.Concatenate(name='multi_scale_concat')([b_k1, b_k3, b_k5])  # (11, 192)
    x = SqueezeExcitation(channels=192, name='se_block')(x)
    x = layers.Conv1D(128, 3, padding='same',
                      kernel_regularizer=regularizers.l2(1e-4), name='conv_deep_1')(x)
    x = layers.BatchNormalization(name='bn_deep_1')(x)
    x = layers.Activation('relu', name='relu_deep_1')(x)
    x = layers.SpatialDropout1D(0.20, name='spatial_drop_1')(x)  # drops entire feature maps


    x = layers.Conv1D(256, 3, padding='same',
                      kernel_regularizer=regularizers.l2(1e-4), name='conv_deep_2')(x)
    x = layers.BatchNormalization(name='bn_deep_2')(x)
    x = layers.Activation('relu', name='relu_deep_2')(x)
    x = layers.GlobalMaxPooling1D(name='global_max_pool')(x)

    x = layers.Dense(256, kernel_regularizer=regularizers.l2(1e-4), name='dense_head_1')(x)
    x = layers.BatchNormalization(name='bn_head_1')(x)
    x = layers.Activation('relu', name='relu_head_1')(x)
    x = layers.Dropout(0.30, name='drop_head_1')(x)

    x = layers.Dense(128, kernel_regularizer=regularizers.l2(1e-4), name='dense_head_2')(x)
    x = layers.BatchNormalization(name='bn_head_2')(x)
    x = layers.Activation('relu', name='relu_head_2')(x)
```

```
    x = layers.Dropout(0.20, name='drop_head_2')(x)

    x = layers.Dense(64, activation='relu', name='dense_head_3')(x)
    out = layers.Dense(1, activation='sigmoid', name='output')(x)

    model = keras.Model(inputs=inp, outputs=out, name='MultiScaleCNN')
    model.compile(
        optimizer=keras.optimizers.AdamW(learning_rate=lr, weight_decay=1e-4),
        loss=BinaryCrossentropy(label_smoothing=0.1),
        metrics=[
            'accuracy',
            keras.metrics.AUC(name='auc'),
            keras.metrics.Precision(name='precision'),
            keras.metrics.Recall(name='recall'),
        ]
    )
    return model

cnn_model = build_multiscale_cnn(INPUT_DIM, lr=5e-4)
cnn_model.summary()
print(f'\nTotal trainable parameters: {cnn_model.count_params():,}')
```

### 3.2.6) Implementation of the LSTM Models on the Comprehensive Dataset

```
def build_lstm_model(input_shape, num_classes=2, lr=1e-3):
    model = keras.Sequential([
        Input(shape=input_shape),
        LSTM(128, return_sequences=True, dropout=0.25, recurrent_dropout=0.1),
        LSTM(64, dropout=0.25, recurrent_dropout=0.1),
        Dense(64, activation='relu'),
        Dropout(0.3),
        Dense(num_classes, activation='softmax')
    ])

  model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy', keras.metrics.AUC(name='auc')]
    )
    return model
```

```
lstm_model = build_lstm_model(input_shape=X_train_lstm.shape[1:], num_classes=2, lr=1e-3)
model = lstm_model
lstm_model.summary()
print(f'\nTotal trainable parameters: {lstm_model.count_params():,}')



history = lstm_model.fit(
    X_train_lstm,
    y_train,
    validation_split=0.20,
    epochs=TOTAL_EPOCHS,
    batch_size=32,
    class_weight=class_weight,
    callbacks=train_callbacks,
    verbose=1
)
```

### 3.2.7) Implementation of the CNN-LSTM Models on the Comprehensive Dataset

```
def build_paper_cnn_lstm(input_shape, num_classes=2, lr=1e-4):
    inputs = Input(shape=input_shape, name='input_2d')
    steps = input_shape[0] * input_shape[1]
    channels = input_shape[2]
    x = layers.Reshape((steps, channels), name='to_sequence')(inputs)

    x = Conv1D(32, kernel_size=3, padding='same', activation='relu', name='conv1')(x)
    x = BatchNormalization(name='bn1')(x)
    x = MaxPooling1D(pool_size=2, padding='same', name='pool1')(x)

    x = Conv1D(64, kernel_size=3, padding='same', activation='relu', name='conv2')(x)
    x = BatchNormalization(name='bn2')(x)
    x = MaxPooling1D(pool_size=2, padding='same', name='pool2')(x)

    x = Conv1D(128, kernel_size=3, padding='same', activation='relu', name='conv3')(x)
    x = BatchNormalization(name='bn3')(x)
    x = MaxPooling1D(pool_size=2, padding='same', name='pool3')(x)

    x = Conv1D(256, kernel_size=3, padding='same', activation='relu', name='conv4')(x)
    x = BatchNormalization(name='bn4')(x)
    x = Dropout(0.20, name='cnn_dropout')(x)
```

```
    x = LSTM(128, return_sequences=False, name='lstm')(x)
    x = Dense(128, activation='relu', name='fc1')(x)
    x = Dropout(0.20, name='fc_dropout')(x)

   outputs = Dense(num_classes, activation='softmax', name='output')(x)
   model = Model(inputs=inputs, outputs=outputs, name='Paper_CNN_LSTM_HeartDisease')
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

cnn_lstm_model = build_paper_cnn_lstm(input_shape=X_train_2d.shape[1:], num_classes=2,
lr=1e-4)
cnn_lstm_model.summary()
print(f"\nTotal trainable parameters: {cnn_lstm_model.count_params():,}")
```

# Chapter 4

# OUTCOMES

## 4.1)Dataset-1 (Cleveland):

```
Loaded: 304 rows | Classes: {0: 164, 1: 140}
Features after engineering: 18 | NaNs: 0 ✅
Selected: ['sex', 'cp', 'chol', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'age_group', 'cv_risk', 'cp_thalach', 'st_severity']

Train: 258 | Test: 46
SMOTE → {1: np.int64(139), 0: np.int64(139)}
Class weights: {0: 1.0, 1: 1.0}

✅ Preprocessing done | Train: (278, 13) | Test: (46, 13)
```

**Fig. 4.1.1 Dataset-1 Overview**

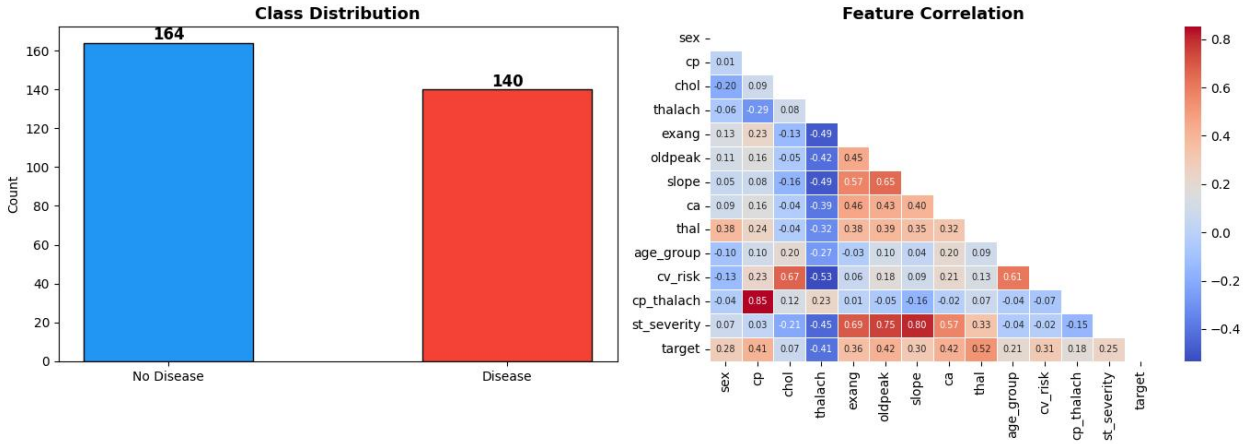



**Fig 4.1.2 Class Distribution and Feature Correlation**

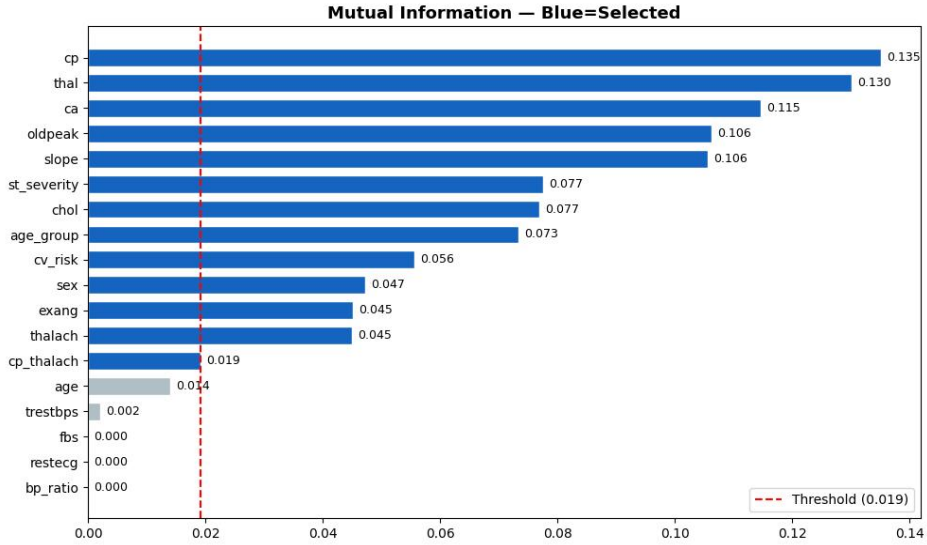
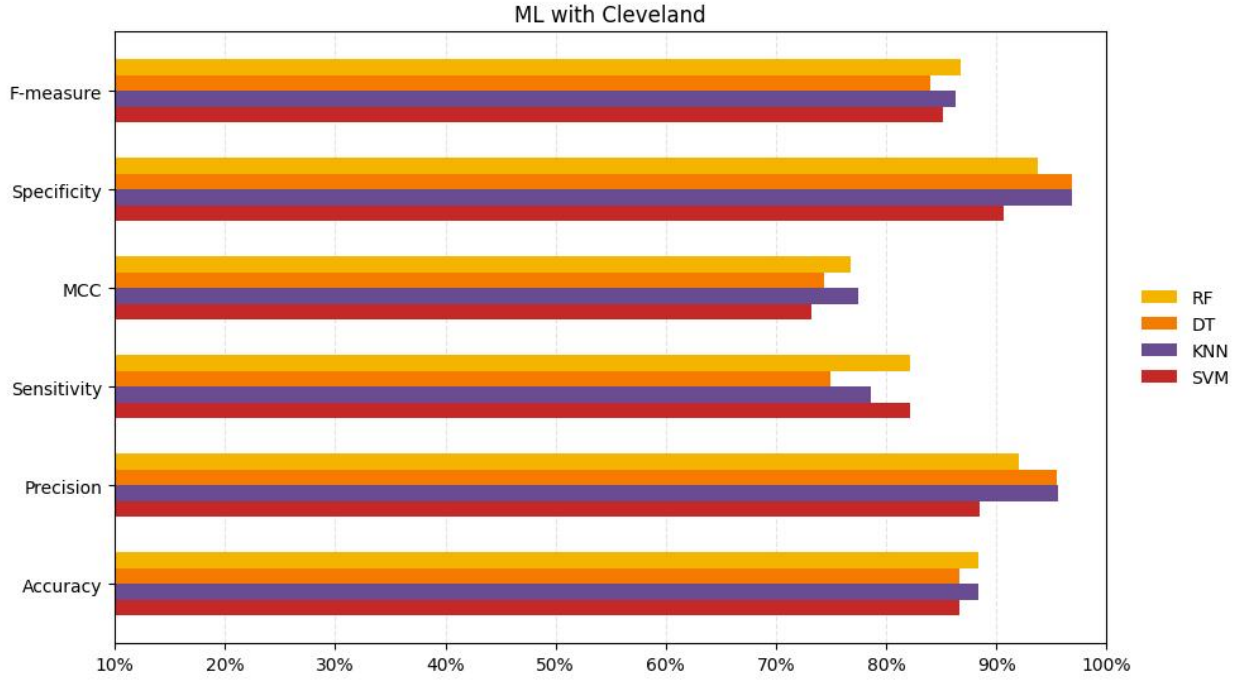


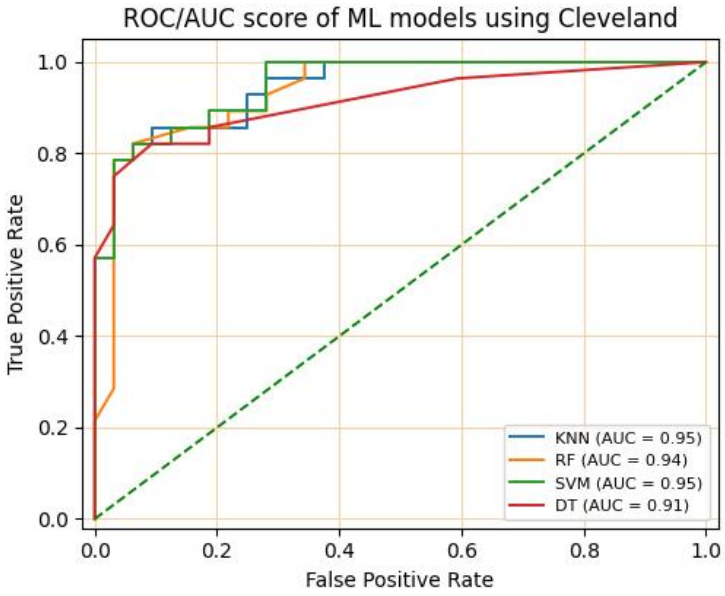

**Fig. 4.1.3 Mutual Information Scores**

**Results:**

- **ML Models**

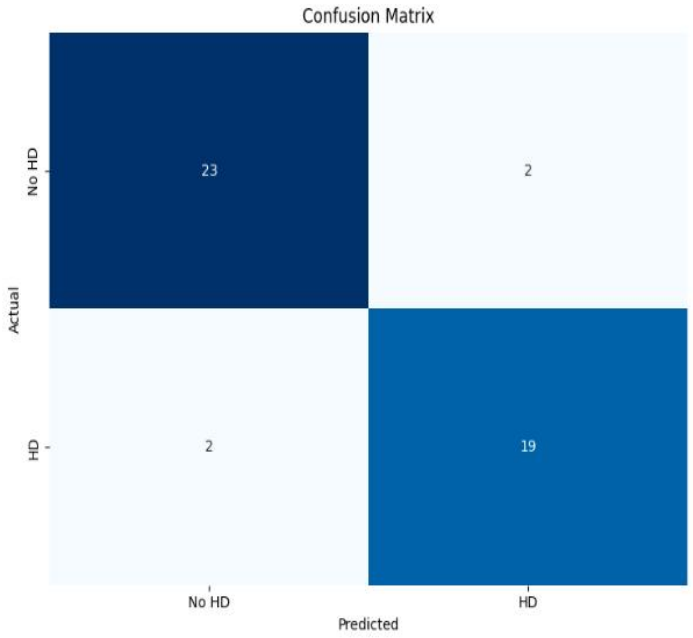
| | Accuracy | Precision | Sensitivity | MCC | Specificity | F-measure |
|---|---|---|---|---|---|---|
| RF | 0.8833 | 0.9200 | 0.8214 | 0.7680 | 0.9375 | 0.8679 |
| DT | 0.8667 | 0.9545 | 0.7500 | 0.7441 | 0.9688 | 0.8400 |
| KNN | 0.8833 | 0.9565 | 0.7857 | 0.7742 | 0.9688 | 0.8627 |
| SVM | 0.8667 | 0.8846 | 0.8214 | 0.7326 | 0.9062 | 0.8519 |

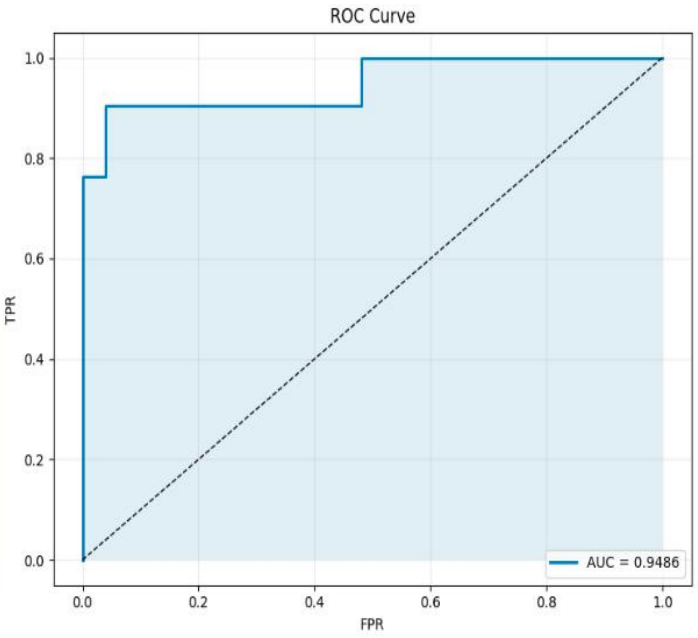


- **ANN**

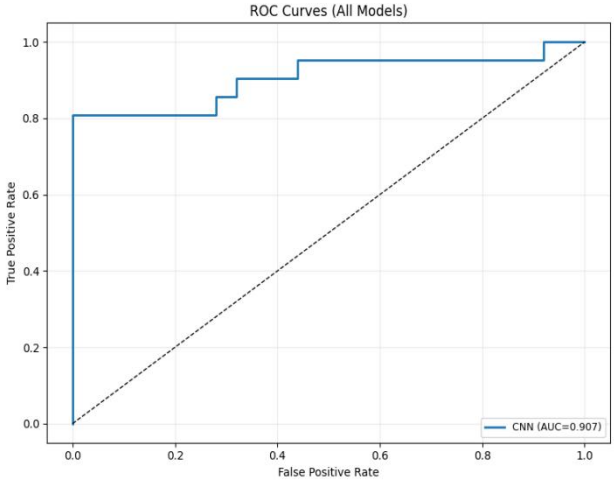
```
Running 50-seed ANN ensemble...
  After 10 seeds: ensemble=91.30%

==================================================
  ANN (50-seed Ensemble, paper arch)
==================================================
  Accuracy : 91.30%
  AUC-ROC  : 0.9486

  Confusion Matrix:
[[23  2]
 [ 2 19]]

  Classification Report:
              precision    recall  f1-score   support

  No Disease       0.92      0.92      0.92        25
     Disease       0.90      0.90      0.90        21

    accuracy                           0.91        46
   macro avg       0.91      0.91      0.91        46
weighted avg       0.91      0.91      0.91        46
```

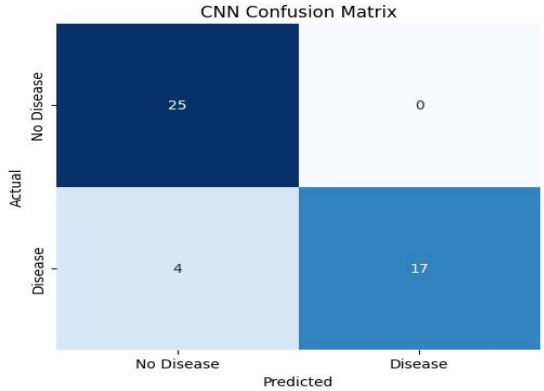


- **CNN**

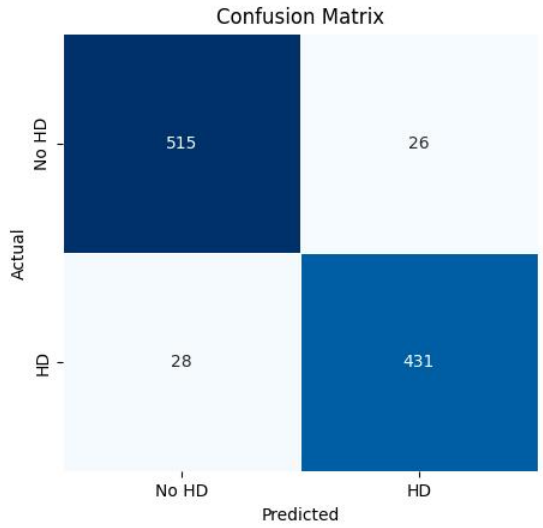
```
==================================
CNN Ensemble (threshold=0.34)
==================================
Accuracy : 91.30%
AUC-ROC  : 0.9067

Confusion Matrix
[[25  0]
 [ 4 17]]

Classification Report
              precision    recall  f1-score   support

           0       0.86      1.00      0.93        25
           1       1.00      0.81      0.89        21

    accuracy                           0.91        46
   macro avg       0.93      0.90      0.91        46
weighted avg       0.93      0.91      0.91        46
```

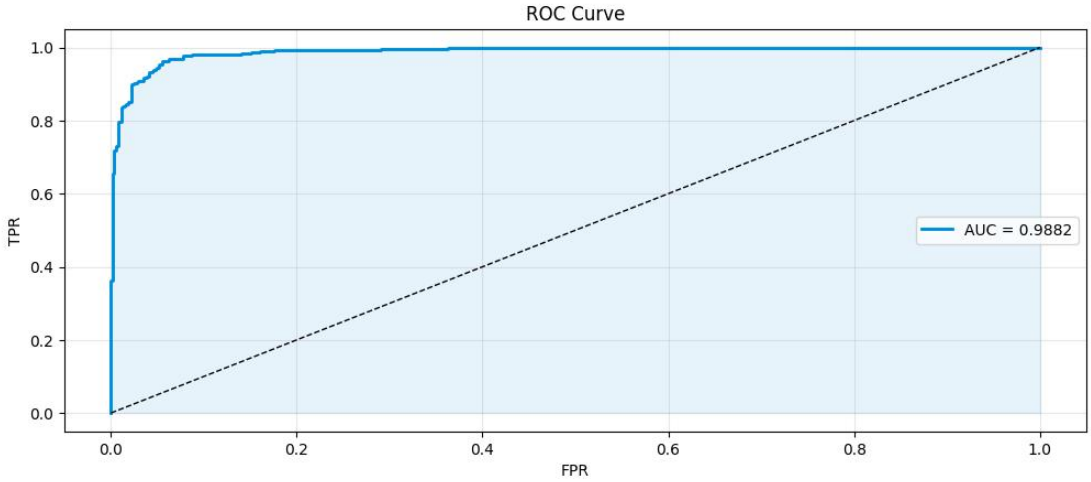


- **LSTM**

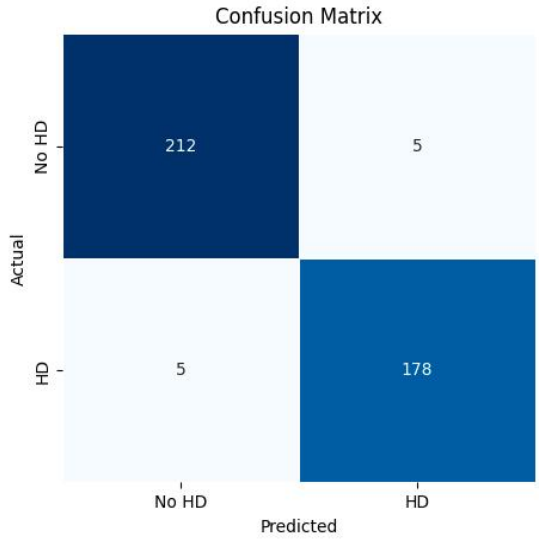
```
======================================================
LSTM — FINAL EVALUATION RESULTS
======================================================
  Accuracy    : 94.60%
  AUC-ROC     : 0.9882
  F1-Score    : 0.9410
  MCC         : 0.8912
  Sensitivity : 0.9390
  Specificity : 0.9519
  Precision   : 0.9431

Confusion Matrix:
[[515  26]
 [ 28 431]]

              precision    recall  f1-score   support

       No HD       0.95      0.95      0.95       541
          HD       0.94      0.94      0.94       459

    accuracy                           0.95      1000
   macro avg       0.95      0.95      0.95      1000
weighted avg       0.95      0.95      0.95      1000
```

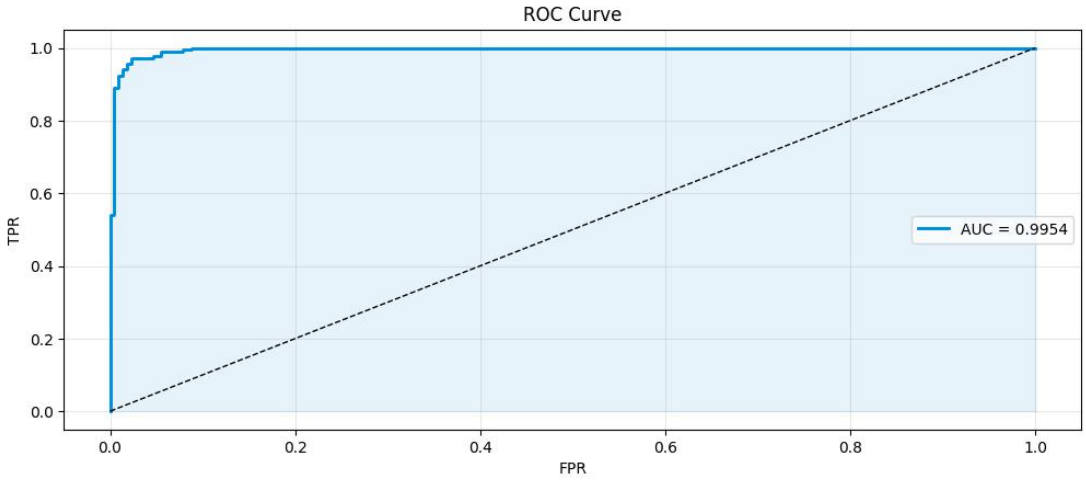


- **CNN-LSTM**

```
======================================================
CNN-LSTM — FINAL EVALUATION RESULTS
======================================================
  Accuracy    : 97.50%
  AUC-ROC     : 0.9954
  F1-Score    : 0.9727
  MCC         : 0.9496
  Sensitivity : 0.9727
  Specificity : 0.9770
  Precision   : 0.9727

Confusion Matrix:
[[212   5]
 [  5 178]]

              precision    recall  f1-score   support

       No HD       0.98      0.98      0.98       217
          HD       0.97      0.97      0.97       183

    accuracy                           0.97       400
   macro avg       0.97      0.97      0.97       400
weighted avg       0.97      0.97      0.97       400
```



## 4.2)Dataset 2( Comprehensive Dataset):

Dataset shape : (1190, 12)
Class distribution: {np.int64(0): np.int64(561), np.int64(1): np.int64(629)}

| | ST slope | chest pain type | exercise angina | max heart rate | cholesterol | oldpeak | age | resting bp s | sex | resting ecg | fasting blood sugar | target |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 0 | 172 | 289 | 0.0 | 40 | 140 | 1 | 0 | 0 | 0 |
| 1 | 2 | 3 | 0 | 156 | 180 | 1.0 | 49 | 160 | 0 | 0 | 0 | 1 |
| 2 | 1 | 2 | 0 | 98 | 283 | 0.0 | 37 | 130 | 1 | 1 | 0 | 0 |
| 3 | 2 | 4 | 1 | 108 | 214 | 1.5 | 48 | 138 | 0 | 0 | 0 | 1 |
| 4 | 1 | 3 | 0 | 122 | 195 | 0.0 | 54 | 150 | 1 | 0 | 0 | 0 |

**Fig 4.2.1 Dataset-2 Overview**

```
Dataset shape: (1190, 12)

Target distribution:
target
1    629
0    561
Name: count, dtype: int64

--------------------------------------
FEATURE IMPORTANCE ANALYSIS
--------------------------------------

Feature Importance Rankings:
 1. ST slope            : 0.1542
 2. exercise angina     : 0.1435
 3. chest pain type     : 0.1337
 4. max heart rate      : 0.0942
 5. cholesterol         : 0.0921
 6. oldpeak             : 0.0888
 7. age                 : 0.0800
 8. resting bp s        : 0.0792
 9. sex                 : 0.0587
10. resting ecg         : 0.0400
11. fasting blood sugar : 0.0356

Training set: (952, 11)
Testing set: (238, 11)
```

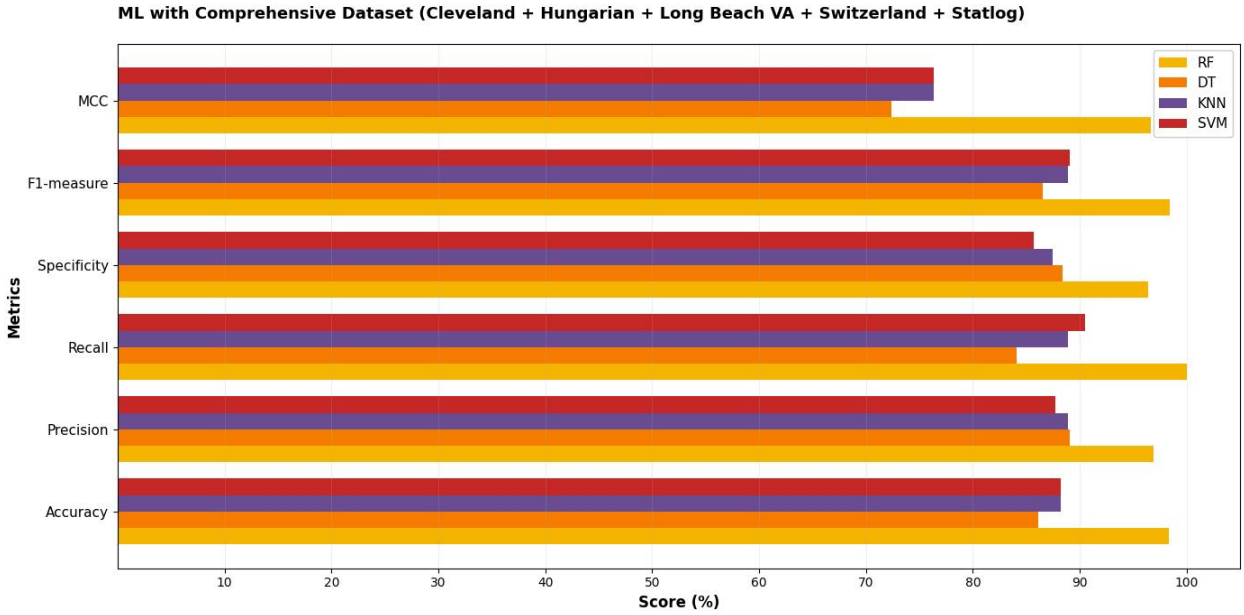
**Fig 4.2.2 Class Distribution and Feature Correlation**

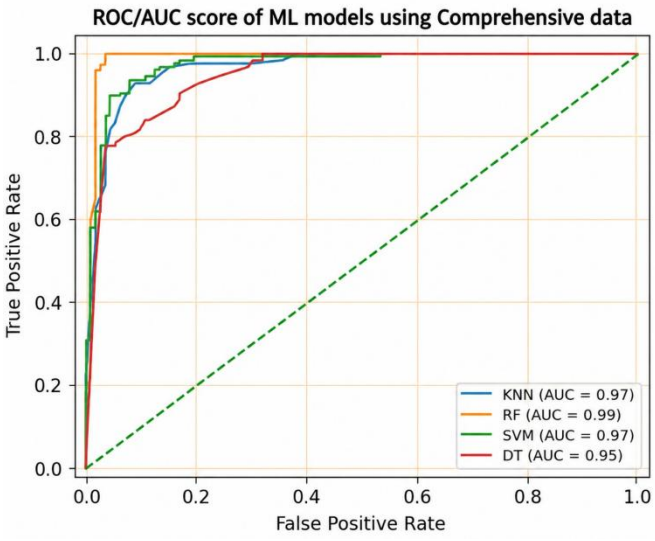
## Results:

- **ML Models**

Comparison Summary (Percent Values):

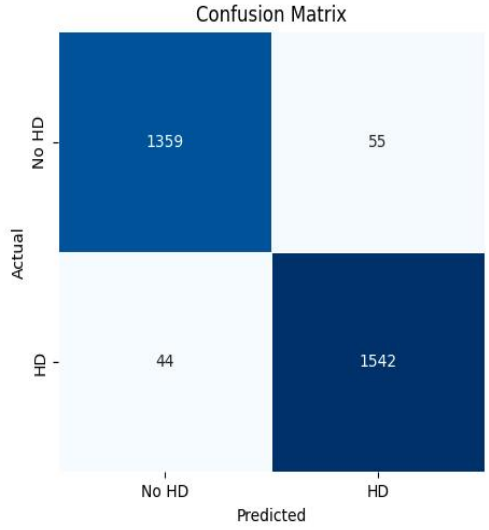
| | Accuracy | Precision | Recall | Specificity | F1-measure | MCC |
|---|---|---|---|---|---|---|
| RF | 98.32 | 96.92 | 100.00 | 96.43 | 98.44 | 96.68 |
| DT | 86.13 | 89.08 | 84.13 | 88.39 | 86.53 | 72.39 |
| KNN | 88.24 | 88.89 | 88.89 | 87.50 | 88.89 | 76.39 |
| SVM | 88.24 | 87.69 | 90.48 | 85.71 | 89.06 | 76.39 |

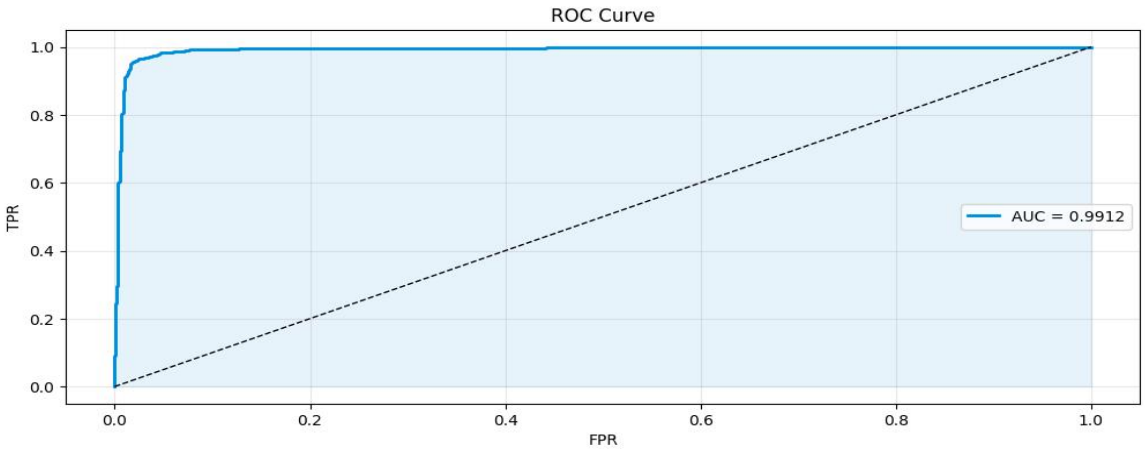


- **ANN**

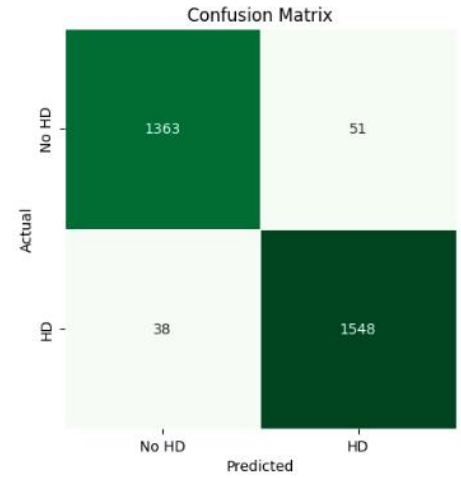
```
======================================================
DEEP ANN — FINAL EVALUATION RESULTS
======================================================
  Accuracy    : 96.70%
  AUC-ROC     : 0.9912
  F1-Score    : 0.9689
  MCC         : 0.9338
  Sensitivity : 0.9723
  Specificity : 0.9611
  Precision   : 0.9656

Confusion Matrix:
[[1359   55]
 [  44 1542]]

              precision    recall  f1-score   support

       No HD       0.97      0.96      0.96      1414
          HD       0.97      0.97      0.97      1586

    accuracy                           0.97      3000
   macro avg       0.97      0.97      0.97      3000
weighted avg       0.97      0.97      0.97      3000
```

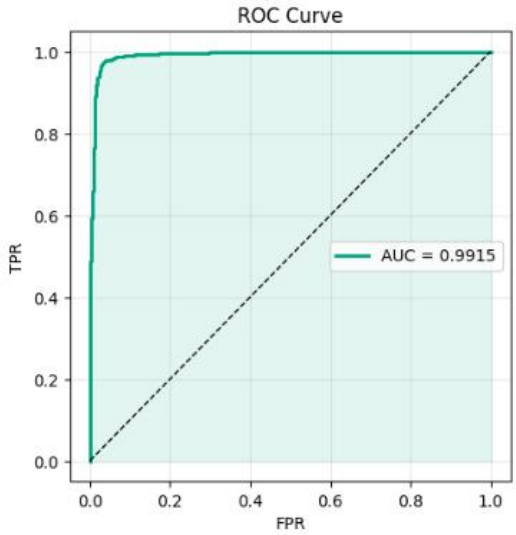


- **CNN**

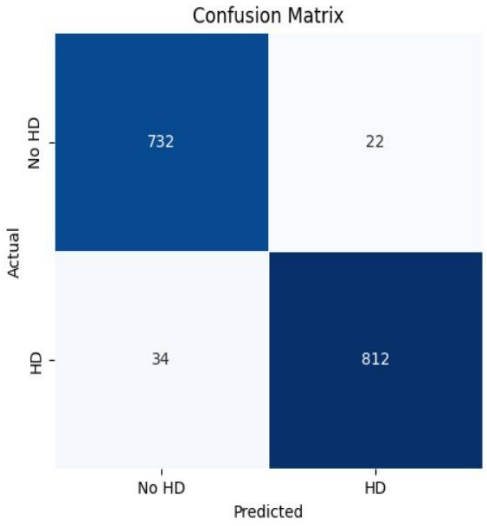
```
======================================================
MULTI-SCALE CNN — FINAL EVALUATION RESULTS
======================================================
  Accuracy    : 97.03%
  AUC-ROC     : 0.9915
  F1-Score    : 0.9721
  MCC         : 0.9405
  Sensitivity : 0.9760
  Specificity : 0.9639
  Precision   : 0.9681

Confusion Matrix:
[[1363   51]
 [  38 1548]]

              precision    recall  f1-score   support

       No HD       0.97      0.96      0.97      1414
          HD       0.97      0.98      0.97      1586

    accuracy                           0.97      3000
   macro avg       0.97      0.97      0.97      3000
weighted avg       0.97      0.97      0.97      3000
```

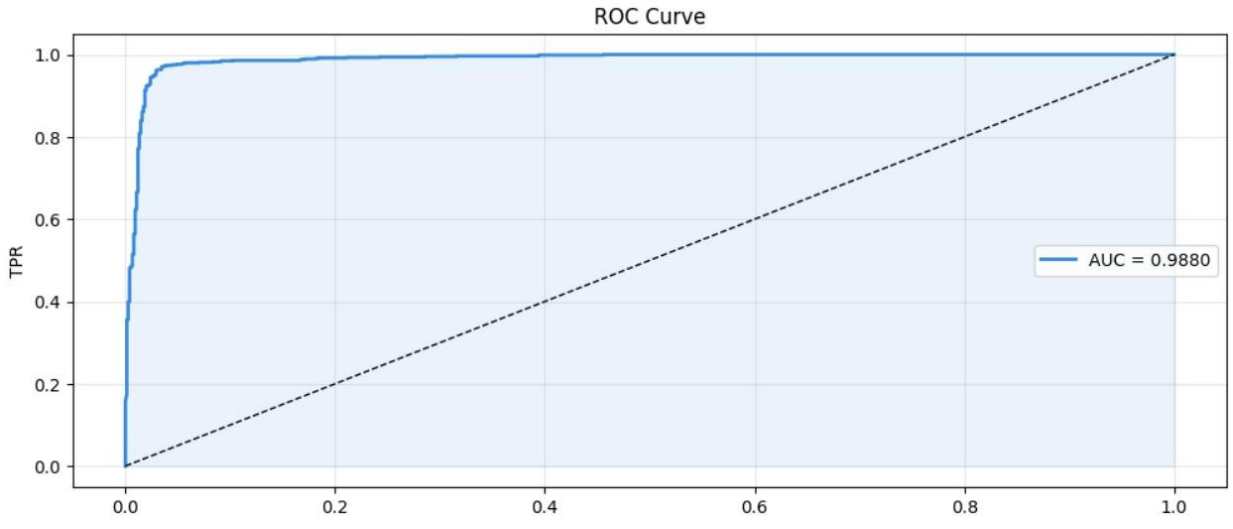


- **LSTM**

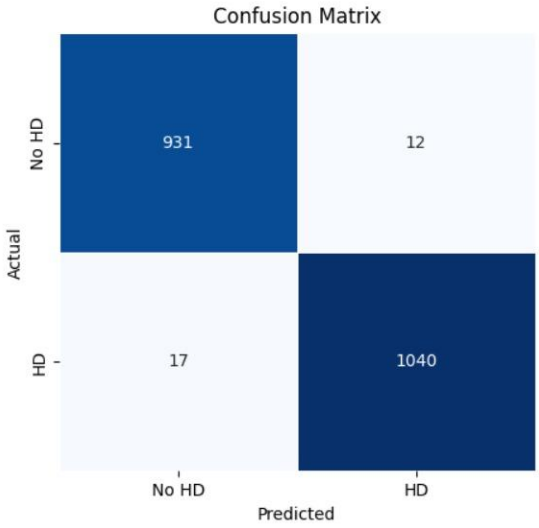
```
======================================================
LSTM — FINAL EVALUATION RESULTS
======================================================
  Accuracy    : 96.50%
  AUC-ROC     : 0.9880
  F1-Score    : 0.9667
  MCC         : 0.9299
  Sensitivity : 0.9598
  Specificity : 0.9708
  Precision   : 0.9736

Confusion Matrix:
[[732  22]
 [ 34 812]]

              precision    recall  f1-score   support

       No HD       0.96      0.97      0.96       754
          HD       0.97      0.96      0.97       846

    accuracy                           0.96      1600
   macro avg       0.96      0.97      0.96      1600
weighted avg       0.97      0.96      0.97      1600
```

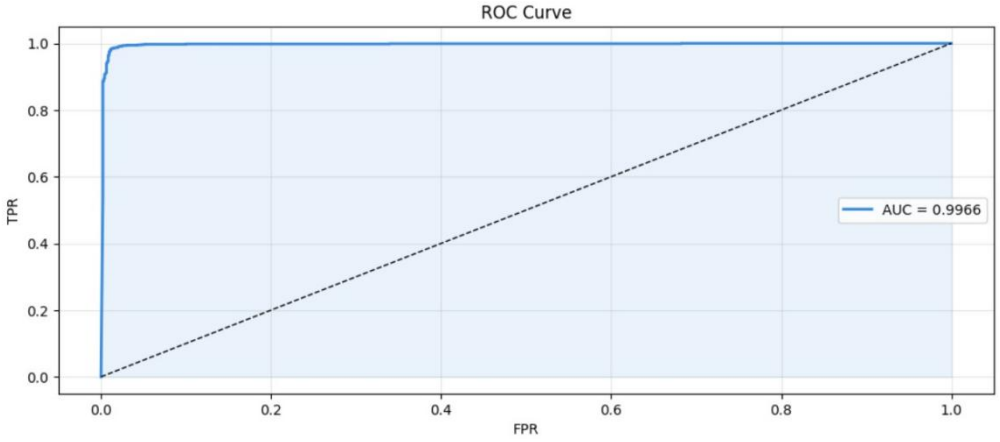


- **CNN-LSTM**

```
======================================================
CNN-LSTM — FINAL EVALUATION RESULTS
======================================================
  Accuracy    : 98.55%
  AUC-ROC     : 0.9966
  F1-Score    : 0.9862
  MCC         : 0.9709
  Sensitivity : 0.9839
  Specificity : 0.9873
  Precision   : 0.9886

Confusion Matrix:
[[ 931   12]
 [  17 1040]]

              precision    recall  f1-score   support

       No HD       0.98      0.99      0.98       943
          HD       0.99      0.98      0.99      1057

    accuracy                           0.99      2000
   macro avg       0.99      0.99      0.99      2000
weighted avg       0.99      0.99      0.99      2000
```



## 4.3)Comparision Tables:

| | BASE PAPER RESULTS | | | | | | | IMPLEMENTATION RESULTS | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **MODEL** | ACCURACY | PRECISION | SENSITIVITY | F1-SCORE | MCC | AUC | SPECIFICITY | ACCURACY | PRECISION | SENSITIVITY | F1-SCORE | MCC | AUC | SPECIFICITY |
| **SVM** | 87% | 89.47% | 90.83% | 90.15% | 77.90% | 87.91% | 86.91% | 86.67% | 88.46% | 82.14% | 86.79% | 73.26% | 95.00% | 90.06% |
| **KNN** | 89% | 88.8% | 91.6% | 90.2% | 77.8% | 88.8% | 85.9% | 88.33% | 95.65% | 78.57% | 86.27% | 77.42% | 95.00% | 96.88% |
| **DECISION TREE** | 87% | 88.80% | 87.8% | 89.6% | 77.5% | 86.03% | 87.0% | 86.67% | 95.45% | 75.00% | 84.0% | 74.41% | 91.00% | 96.88% |
| **RANDOM FOREST** | 91% | 93.81% | 91.91% | 92.85% | 84.23% | 91.20% | 92.50% | 88.33% | 92% | 82.14% | 86.79% | 76.80% | 94.00% | 93.75% |

**Table 4.1 Comparison of ML models Over Cleveland Dataset**

| | BASE PAPER RESULTS | | | | | | | IMPLEMENTATION RESULTS | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **MODEL** | ACCURACY | PRECISION | SENSITIVITY | F1-SCORE | MCC | AUC | SPECIFICITY | ACCURACY | PRECIS ION | SENSITIVITY | F1-SCORE | MCC | AUC | SPECIFICITY |
| **SVM** | 89.07% | 86.71% | 82.24% | 90.05% | 78.11% | 88.44% | 82.24% | 88.24% | 87.69% | 90.48% | 89.06% | 76.39% | 97.00% | 85.71% |
| **KNN** | 88.65% | 87.14% | 93.212% | 90.0% | 77.12% | 88.15% | 88.15% | 88.24% | 88.89% | 88.89% | 88.89% | 76.39% | 97.00% | 87.50% |
| **DECISION TREE** | 86.34% | 89.5% | 85.79% | 87.61% | 72.23% | 83.31% | 86.82% | 86.13% | 89.08% | 84.13% | 86.53% | 72.39% | 95.00% | 88.39% |
| **RANDOM FOREST** | 89.93% | 91.12% | 91.12% | 91.12% | 79.49% | 89.74% | 88.37% | 98.32% | 96.92% | 100.00% | 98.44% | 96.68% | 99.00% | 96.43% |

**Table 4.2 Comparison of ML models Over Comprehensive dataset**

| | BASE PAPER RESULTS | | | | | | | IMPLEMENTATION RESULTS | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **MODEL** | ACCURACY | PRECISION | SENSITIVITY | F1-SCORE | MCC | AUC | SPECIFICITY | ACCURACY | PRECISION | SENSITIVITY | F1-SCORE | MCC | AUC | SPECIFICITY |
| **ANN** | 93.21% | 94.08% | 94.08% | 94.08% | 86.29 % | 95.14% | 92.20 % | 91.30% | 90.48% | 90.48 % | 90.48 % | 82.48% | 94.86 % | 92.00% |
| **CNN** | 95.02 % | 95.21 % | 95.46% | 95.63% | 94.95 % | 96.86% | 95.71 % | 91.30% | 94.20% | 93.50 % | 94.89 % | 93.22% | 94.48 % | 94.44% |
| **LSTM** | 94.65% | 94.65% | 91.91% | 95.33% | 90.03 % | 95.78% | 94.41 % | 94.60% | 94.31% | 93.90 % | 94.10 % | 89.12% | 98.82 % | 95.19% |
| **CNN-LSTM** | 97.75% | 98.57% | 98.87% | 97.18% | 96.60 % | 98.85% | 97.87 % | 97.50% | 97.27% | 97.27 % | 97.27 % | 94.96% | 99.54 % | 97.70% |

**Table 4.3 Comparison of DL models Over Cleveland Dataset**

| | BASE PAPER RESULTS | | | | | | | IMPLEMENTATION RESULTS | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **MODEL** | ACCURACY | PRECISION | SENSITIVITY | F1-SCORE | MCC | AUC | SPECIFICITY | ACCURACY | PRECISION | SENSITIVITY | F1-SCORE | MCC | AUC | SPECIFICITY |
| **ANN** | 94.53% | 94.02% | 96.18% | 95.09% | 88.96% | 96.85 % | 92.52% | 96.70% | 96.56% | 97.30% | 96.89% | 93.38% | 99.12% | 96.11% |
| **CNN** | 96.86% | 97.22% | 97.22% | 97.22% | 92.96% | 98.78 % | 95.78% | 97.03% | 96.81% | 97.6% | 97.21% | 94.05% | 99.15% | 96.39% |
| **LSTM** | 96.64% | 97.93% | 95.95% | 96.93% | 93.25% | 98.65 % | 97.50% | 96.50% | 97.36% | 95.98% | 96.67% | 92.99% | 98.80% | 97.08% |
| **CNN-LSTM** | 98.86% | 99.13% | 98.78% | 99.83% | 97.05% | 99.78 % | 99.42% | 98.55% | 98.86% | 98.39% | 98.62% | 97.90% | 99.66% | 98.73% |

**Table 4.4 Comparison of DL models Over Comprehensive Dataset**

# CHAPTER 5
# CONCLUSION AND FUTURE PLANS

The main objective of this research work was to develop a reliable prediction system for predicting heart diseases using hybrid deep neural networks. For this purpose, different approaches for implementing machine learning have been used, including Decision tree, SVM, KNN, random forest, ANN, CNN, and LSTM algorithms. In addition, a hybrid approach was also used which comprised both CNN and LSTM. The analysis of these algorithms in respect of their accuracy, precision, sensitivity, specificity, F1-score, MCC, and AUC.

It is clear from the above result that the hybrid CNN-LSTM algorithm gave the best results among all other algorithms. This technique gives us accuracy of around 98.86% when trained on training as well as testing data together. This means that the hybrid model learns and makes predictions in heart disease data very efficiently. Therefore, using a combination of deep learning techniques provides better results than any other method.

One of the main limitations with the current implementation of the model is the lack of capacity to conduct multiclass classification. It should be noted that at present the current approach can determine only the presence of heart disease or absence thereof. Future development may include an opportunity to detect different types and stages of heart disease instead of determining whether someone has heart problems or not. Moreover, use of information derived from actual patients' records stored in hospitals can increase the reliability of the algorithm significantly.

It is also possible to mention the improvement of the model performance. For instance, in the future we can test methods of ensemble learning, which involve use of not a single but multiple algorithms to achieve certain goals. Moreover, increased emphasis can be placed on the data preprocessing phase, since high-quality information always improves performance of the model significantly. Such changes could result in increased efficiency and improved adaptability of the algorithm.

The model under discussion can find practical application in clinical environment. It can facilitate detection of heart disease and support doctors in the decision-making process regarding choosing treatment.

# CHAPTER 6
# REFERENCES


[1] M. S. Al Reshan, S. Amin, M. A. Zeb, A. Sulaiman, H. Alshahrani, and A. Shaikh, "A robust heart disease prediction system using hybrid deep neural networks," *IEEE Access*, vol. 11, pp. 1–18, Oct. 2023.

[2] UCI Machine Learning Repository, "Cleveland heart disease dataset," University of California, Irvine, 1988.

[3] IEEE Data Port / Kaggle, "Combined heart disease dataset (Switzerland, Cleveland, Statlog, Hungarian, Long Beach VA)," 2020.

[4] F. Pedregosa et al. "Scikit-learn: Machine learning in Python," J. *Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[5] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," Google, 2015.

[6] S. Hochreiter and J. Schmidhuber, *Long short-term memory*, Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997.

[7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, *ImageNet classification with deep convolutional neural networks*, Adv. Neural Inf. Process. Syst., 2012.

[8] R. Gupta et al., *Predicting heart disease using IoT-enabled deep learning and machine learning techniques*, Springer LNNS, 2023.

[9] A. Tursynova et al., *Deep learning-enabled heart disease classification using clinical data*, Comput. Mater. Contin., 2023.

[10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proc. *IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[11] M.Abdar, M. Zomorodi-Moghadam, R. Das, and I. Ting, *Performance analysis of classification algorithms on early detection of heart disease*, Expert Syst. Appl., vol. 117, pp. 346–353, 2019.

[12] S. Mohan, C. Thirumalai, and G. Srivastava, *Effective heart disease prediction using hybrid machine learning techniques*, IEEE Access, vol. 7, pp. 81542–81554, 2019.

# CHAPTER 7

# APPENDIX

**Base Paper:**

Al Reshan, M. S.; Amin, S.; Zeb, M. A.; et al.,"***A Robust Heart Disease Prediction System Using Hybrid Deep Neural Networks",***IEEE Access, 2023.

**Keywords:**

Heart disease prediction, Cardiovascular disease (CVD), Deep learning, Hybrid deep neural networks (HDNN), CNN-LSTM, ANN, feature selection, medical datasets, classification, machine learning, healthcare analytics, early diagnosis

**Base Paper URL:**

https://doi.org/10.1109/ACCESS.2023.3328909

**Kaggle Dataset Links:**

- Dataset1(ClevelandHeartDiseaseDataset): https://archive.ics.uci.edu/ml/datasets/heart+disease
- Dataset 2 (Combined Dataset – Switzerland, Cleveland, Statlog, Hungarian, Long Beach VA): https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction

**Implementation Environment:**

- **Platform:** Google Colaboratory (cloud-based Jupyter environment)
- **Hardware (optional GPU):** NVIDIA T4 / CPU-based execution
- **Programming Language:** Python 3.10
- **Deep Learning Frameworks:** TensorFlow 2.x, Keras
- **Key Libraries:** scikit-learn, NumPy, Pandas, Matplotlib, Seaborn
- **Additional Tools:** imbalanced-learn (SMOTE), joblib
- **Model Saving Format:** .h5 / .keras(TensorFlow native format)